Front. Comput. Sci., 2025, 19(7): 197605
https://doi.org/10.1007/s11704-024-40663-9

REVIEW ARTICLE

A survey on LoRA of large language models

Yuren MAO[1,2], Yuhang GE[1], Yijiang FAN[1], Wenyi XU[1], Yu MI[1], Zhonghao HU[1], Yunjun GAO (✉)[1,2]

1 School of Software Technology, Zhejiang University, Ningbo 315000, China
2 College of Computer Science and Technology, Zhejiang University, Hangzhou 310007, China

© The Author(s) 2024. This article is published with open access at link.springer.com and journal.hep.com.cn

Abstract Low-Rank Adaptation (LoRA), which updates the dense neural network layers with pluggable low-rank matrices, is one of the best performed parameter efficient fine-tuning paradigms. Furthermore, it has significant advantages in cross-task generalization and privacy-preserving. Hence, LoRA has gained much attention recently, and the number of related literature demonstrates exponential growth. It is necessary to conduct a comprehensive overview of the current progress on LoRA. This survey categorizes and reviews the progress from the perspectives of (1) downstream adaptation improving variants that improve LoRA's performance on downstream tasks; (2) cross-task generalization methods that mix multiple LoRA plugins to achieve cross-task generalization; (3) efficiency-improving methods that boost the computation-efficiency of LoRA; (4) data privacy-preserving methods that use LoRA in federated learning; (5) application. Besides, this survey also discusses the future directions in this field.

Keywords low-rank adaptation, LoRA, large language models, LLMs

1 Introduction

Rapidly increasing parameter scales of pre-training language models improves their generalization ability and brings emergent abilities. In the last few years, the parameter scales of pre-training languages models have increased by thousands of times (e.g., from 330 M parameter BERT [1] to 540 B parameter PaLM [2]). These pre-training language models having large parameter scales are termed Large language models (LLMs). Nevertheless, due to the knowledge boundaries of the LLMs, their abilities on some downstream tasks are still limited. To expand the knowledge boundaries, it remains necessary to fine-tune LLMs on the downstream tasks.

However, fine-tuning the full parameters of an LLM, namely full fine-tuning, is extremely computationally expensive, for example, full fine-tuning of a LLaMA2-7B [3] model requires approximately 60 GB of memory, which exceeds the capacity of common consumer GPUs [4]. To reduce the computational cost, various parameter-efficient fine-tuning (PEFT) methods have been proposed [5]. They adapt LLMs to downstream tasks by only fine-tuning a small number of (extra) model parameters. From the perspective of whether extra parameters are involved, PEFT methods can be divided into two categories: extra-parameter methods and intra-parameter methods. The extra-parameter methods freeze all of the original parameters of an LLM and insert a set of learnable parameters to optimize the model input or model layers such as adapter tuning [6] and prompt tuning [7]. By contrast, intra-parameter methods freeze most of the original parameters of an LLM and only tune a small number of parameters of the LLM such as BitFit [8], LISA [4] and LoRA [9].

When we do not have access to modify the model architecture, intra-parameter methods are desirable. Among the intra-parameter methods, LoRA is the most widely used one, because it can achieve a comparable or better downstream adaptation performance to the full fine-tuning on a range of downstream tasks [9] and is easy to implement. Besides, there are many variants have been proposed to further improve the downstream adaptation ability of LoRA on more challenging downstream tasks.

LoRA achieves parameter efficiency by updating the dense neural network layers of an LLM with pluggable low-rank matrices. These matrices (a.k.a, LoRA plugins) are independent of the LLM, which can be stored and reused in other related downstream tasks. Furthermore, these LoRA plugins can be combined to achieve cross-task generalization, which can facilitate multi-task learning, domain adaptation, and continual learning.

As the LoRA modules accumulate, the computation cost of managing LoRA modules is increasing. Although LoRA is computation-efficient, the computational cost of managing a larger number of LoRA modules is unignorable. It is necessary to further improve the computation efficiency of LoRA. The improvement can come from reducing the computation cost of single LoRA modules and accelerating the scalable serving of multiple modules. It can boost the application of LoRA in real-world use cases, such as Generative-as-a-Service (GaaS) cloud products.

In some cases, the training data are privately owned by

Received July 2, 2024; accepted August 12, 2024

E-mail: gaoyj@zju.edu.cn

multiple clients and cannot be centralized. To adapt LLMs with the distributed training data, we can adopt federated learning to protect the data privacy of each client. However, federated learning suffers expensive communication and computation costs. To reduce costs, LoRA is a natural choice. Its parameter-efficient nature helps to reduce the computation cost of each client and the communication cost of sharing parameters across clients. Furthermore, the pluggable feature of LoRA, which supports the localization or encryption of personalized parameters, enhances privacy protection within federated learning. Therefore, LoRA has a great potential for privacy-preserving.

While some previous surveys have mentioned LoRA [5,10,11], they mainly focus on PEFT and only introduce a small number of LoRA-related works, lacking systematic treatment and comprehensive overview on LoRA and its variants. In this survey, we give a comprehensive overview of the current progress on LoRA for methods (1) improving downstream adaption performance of LoRA; (2) mixing LoRA modules to achieve cross-task generalization; (3) boosting the computation-efficiency of LoRA; (4) adopting LoRA in federated learning. Besides, the application of LoRA is briefly introduced. This taxonomy of LoRA-related methods is illustrated in Fig. 1. This survey is expected to give comprehensive background knowledge, research trends and technical insights for LoRA.

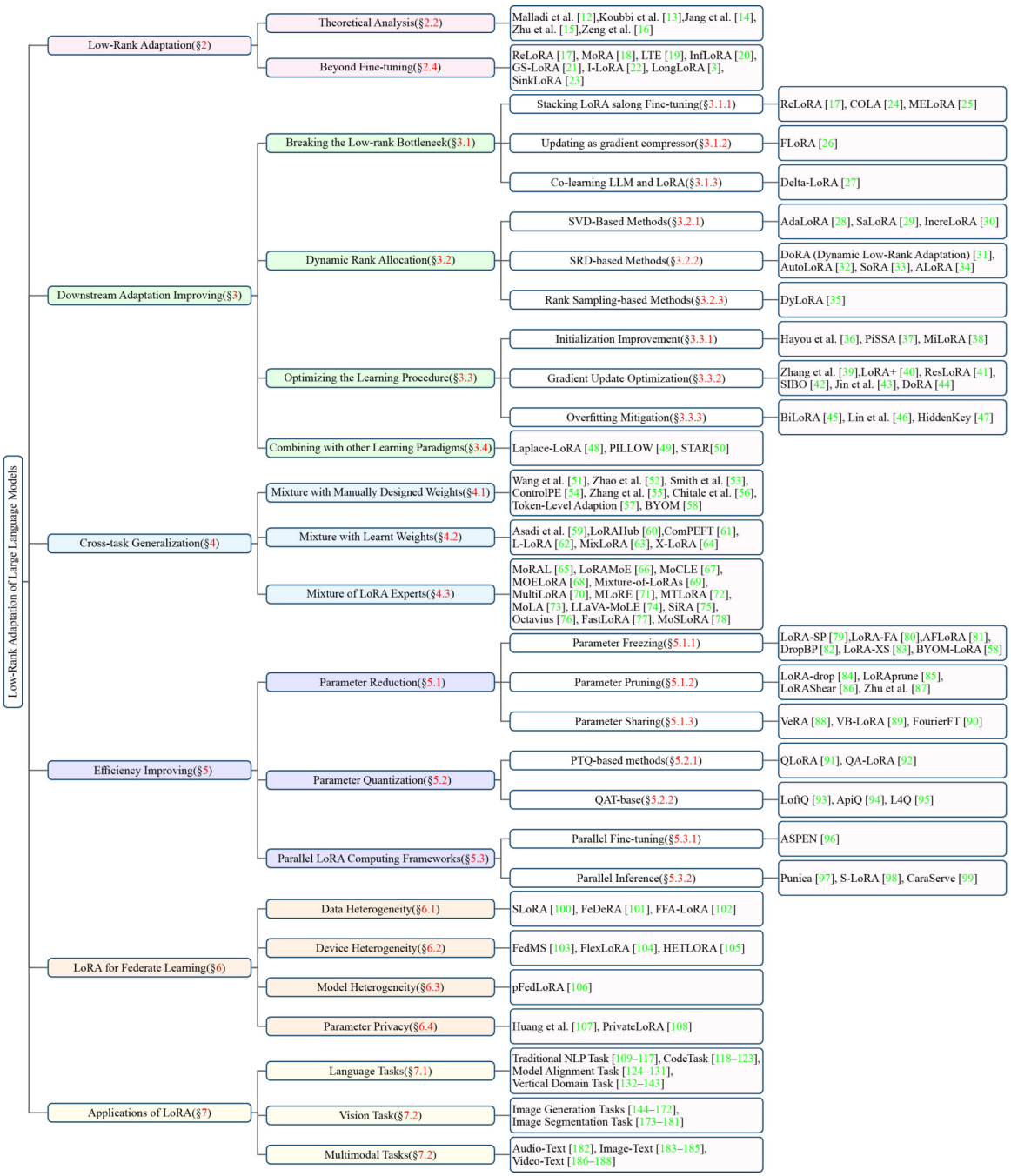


Fig. 1 The taxonomy of this paper

The rest of this survey is organized as follows. Section 2 introduces the background knowledge of LoRA, and Section 3 introduces the LoRA's variants that aim to improve the downstream adaptation performance. In Section 4, we review the LoRA mixture methods that mix LoRA modules to achieve cross-task generalization. Section 5 discusses the methods that are propsoed to improve the computational efficiency of LoRA. The LoRA-driven federated learning methods are introduced in Section 6. Section 7 reports the applications of LoRA. We conclude this survey and discuss the future directions in Section 8.

2 Low-rank adaptation (LoRA)

The Low-dimensional intrinsic dimensionality hypothesis [189] presents that over-parameterized models reside on a low intrinsic dimension, which demonstrates that we can achieve proper learning performance by only updating parameters related to the intrinsic rank. Based on this hypothesis, LoRA [9] proposes to update dense layers in a model with low-rank matrices. It can achieve both parameter- and computational-efficiency. In this section, we first introduce the details of LoRA and then introduce existing works that focus on the theoretical analysis of LoRA. Furthermore, we demonstrate LoRA's efficiency in practice. At last, this section presents that LoRA can be used in other use cases except fine-tuning.

2.1 LoRA

Given a dense neural network layer parameterized by $W_0 \in \mathbb{R}^{d\times k}$, to adapt it to a downstream task, we update it with $\Delta W \in \mathbb{R}^{d\times k}$ and obtain an updated layer parameterized by $W = W_0 + \Delta W$. For full fine-tuning, ΔW is computed based on gradients of all the $d\times k$ parameters for the layer, which is computationally expensive and requires a large amount of GPU memory for LLMs. To improve the computational efficiency, LoRA decomposes ΔW into two small matrices $B \in \mathbb{R}^{d\times r}$ and $A \in \mathbb{R}^{r\times k}$, i.e.,

$$W = W_0 + \alpha BA, \tag{1}$$

where $r \ll min\{d,k\}$, B and A are initialized with a random Gaussian distribution and zero respectively, α represents the scaling factor that controls the strength of updates. The parameter number of LoRA is $r\times(d+k)$, which is significantly less than $d\times k$. Figure 2(a) and (b) compare the structures of full fine-tuning and LoRA.

LoRA is highly **parameter efficient** for it updates only a small subset of model parameters, which reduces the memory and computational requirements for fine-tuning without increasing inference latency [190]. Furthermore, The parameter efficiency can be further improved by extending from the low-rank matrix to low-rank tensor [191] or combining with the Kronecker decomposition [192,193]. Except for parameter efficiency, LoRA is also **pluggable** for the LoRA parameters that can be separated from the model after training. The pluggable character of LoRA enables it to be shared and reused by multiple users [194]. When we have LoRA modules for multiple tasks, we can combine these modules and expect a proper **cross-task generalization** performance [60]. Besides, the low-rank mechanism of LoRA is **compatible** with other parameter-efficient methods, such as adapter [195,196]. Besides, LoRA can achieve **proper downstream adaptation performance** on various downstream tasks. For example, on MMLU [197] benchmark, comparing with full fine-tuning, fine-tuning with LoRA can achieve comparable or even better performance across 57 tasks [4].

In practice, for a Transformer-based LLM, the dense layers typically consist of two types of weight matrices: the projection matrices in attention modules and feed-forward neural (FFN) modules. The experiments mentioned above are conducted based on the original LoRA settings, applying it to the query and value weight matrices in the attention modules. It is worth mentioning that subsequent work shows that applying it to the FFN layers can further improve model performance [198].

2.2 Theoretical analysis

To understand why LoRA is effective and how LoRA can be more effective, several works have provided theoretical analyses from various aspects. To answer the question that why LoRA is effective, Malladi et al. [12] analyze the fine-tuning dynamics of LoRA from the kernel view and demonstrate that in the lazy regime, LoRA fine-tuning is nearly equivalent to full fine-tuning. Besides, Zeng et al. [16] provides a theoretical analysis of the LoRA's expressive power for both fully connected neural networks (FNNs) and Transformer networks (TFNs). They proved that LoRA can adapt any model f to accurately represent any smaller target model $\bar{f}$ if LoRA-rank $\geqslant$ (width of f) $\times \frac{depth\,of\,\bar{f}}{depth\,of\,f}$ under a mild assumption, where the depth and width are the number of

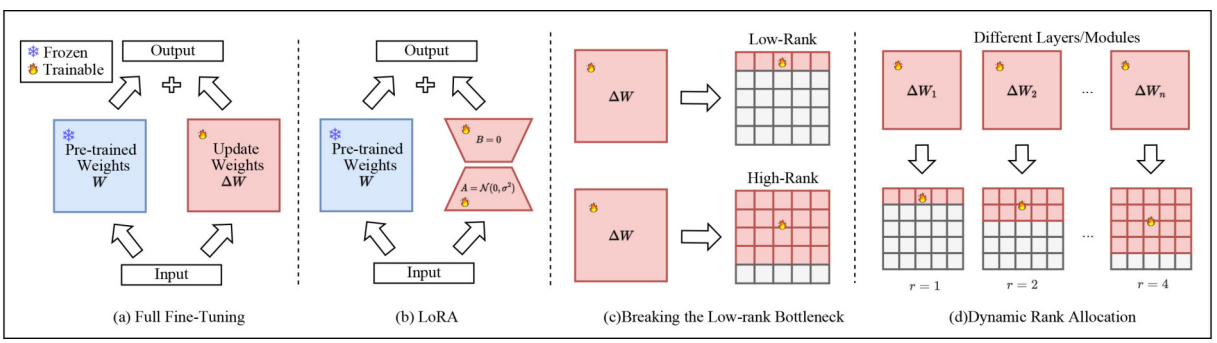


Fig. 2 An illustration of full fine-tuning (a), LoRA (b) and its variants for improving downstream adaptation, which includes breaking the low-rank bottleneck (c) and dynamic rank allocation (d)

layers and the number of neurons of the layer having the largest number of neurons, respectively. Moreover, they quantify the approximation error when the LoRA-rank falls below this threshold. Regarding TFNs, they showed that any model can be adapted to a target model of equivalent size using a rank-$\left(\frac{\text{embedding size}}{2}\right)$ for LoRA. Additionally, Koubbi et al. [13] utilize the mathematical framework for Transformers established by [199–201] to investigate the how low-rank perturbations in attention parameters affect.

As to the question that how LoRA can be more effective, Jang et al. [14] analyze the fine-tuning of LoRA within the neural tangent kernel (NTK) [202] framework when N data points are available. They demonstrate that employing a rank $r \gtrsim \sqrt{N}$ in LoRA helps to avoid spurious local minima and facilitates the discovery of low-rank solutions that exhibit good generalization. Besides, Zhu et al. [15] observe that the project-down matrix A is utilized for extracting features from the input, while the project-up matrix B employs these features to create the desired output. Based on this observation, they demonstrate that freezing the project-down matrix A while tuning only the project-up matrix B leads to better generalization compared to tuning both matrices, in addition to achieving a 2× reduction in parameters.

2.3 Efficiency in practice

The computational efficiency of LoRA is significantly higher than that for full fine-tuning. Taking fine-tuning the dense weight matrix of the first FFN layer in LLaMA2-7B as an example, full fine-tuning needs to fine-tune $11,008 \times 4,096 = 45,088,768$ parameters while LoRA only needs to tune $(11,008 \times 4) + (4 \times 4,096) = 60,416$ parameters when $r = 4$. For this layer, LoRA only adjusts nearly one-thousandth of the parameters compared to full fine-tuning.

LoRA can significantly decrease the memory usage of fine-tuning an LLM, which can be divided into four parts: (1) Model Memory: the memory required to store the model weights; (2) Activation Memory: the memory occupied by intermediate activations during forward propagation. It mainly depends on factors such as batch size and sequence length; (3) Gradient Memory: the memory required to store gradients during backpropagation. The gradients are only calculated for trainable parameters; (4) Optimization Memory: the memory used to store optimizer states. For example, the Adam optimizer stores the "first moment" and "second moment" of trainable parameters.

Pan et al. [4] provides a comprehensive empirical comparison between full fine-tuning and LoRA fine-tuning on an LLaMA2-7B model with batch size 1, utilizing a single NVIDIA RTX4090 (24 GB) GPU. According to this study, full fine-tuning requires approximately 60 GB of memory, which exceeds the capacity of an RTX4090 GPU; by contrast, LoRA fine-tuning only needs about 23 GB of memory. LoRA significantly reduces memory usage and makes fine-tuning LLaMA2-7B feasible on a single NVIDIA RTX4090 (24 GB) GPU. Specifically, due to fewer trainable parameters, both optimization memory and gradient memory decrease significantly by approximately 25 GB and 14 GB respectively. On the other hand, while LoRA introduces additional "incremental parameters" resulting in slight increases in activation memory and weight memory (totaling about 2 GB), this increase is negligible when considering the overall reduction in memory. Moreover, reducing memory brings an acceleration of forward propagation. LoRA is 1.9× times faster compared to full fine-tuning.

2.4 Beyond fine-tuning

Besides fine-tuning, LoRA can be applied to other learning paradigms, such as pre-training [17,19] and continual training [20]. For pre-training, **ReLoRA** [17] and **MoRA** [18] are proposed to use low-rank updates to train high-rank networks; moreover, **LTE** [19] is proposed to perform parallel training of multiple low-rank heads across computing nodes to minimize the need for frequent synchronization, which facilitates the utilization of LoRA in pre-training. As for continual training, there are several methods have been proposed to address the catastrophic forgetting problem. **InfLoRA** [20] addresses catastrophic forgetting by reparameterizing pre-trained weights with a minimal set of parameters in a subspace. **GS-LoRA** [21] uses group sparse regularization to automatically select specific LoRA groups while zeroing out others to mitigate catastrophic forgetting effects. **I-LoRA** [22] leverages dual-memory experience replay combined with LoRA parameter interpolation to combat catastrophic forgetting.

Furthermore, LoRA can be used to overcome the limited context size for LLMs [3,23]. For instance, **LongLoRA** [3] successfully computaitional efficiently extends the context window of LLaMA2-7B [203] from 4k to 100k tokens by combining LoRA with shifted sparse attention. However, LongLoRA does not match the efficiency of vanilla attention due to chaotic attention head structures and unnecessary information exchange between token groups. To address these issues, **SinkLoRA** [23] introduces Sink Fixed Attention (SF-Attn) to proportionally returns cyclically shifted groups of attention heads to their un-shifted state and achieves proper performance.

3 Downstream adaptation improving

Although LoRA can achieve proper adaptation performance on some downstream tasks, there is still a performance gap between LoRA and full fine-tuning on many downstream tasks, such as mathematical reasoning [204–206]. To fill this gap, many methods are proposed to further improve the downstream adaptation performance of LoRA. Typically, existing methods improve the downstream adaptation performance from the following perspectives: (1) breaking the low-rank bottleneck, refer to Fig. 2(c); (2) adaptively allocating the ranks of different LoRA modules, refer to Fig. 2(d); (3) optimizing the learning procedure of LoRA; (4) combining with other learning paradigms. In this section, we introduce these four types of methods respectively.

3.1 Breaking the low-rank bottleneck

The low-rank updates enable LoRA to be parameter efficient; however, it restricts LLMs' ability to memorize downstream knowledge and generalization on downstream tasks [18,205–208]. This low-rank limitation causes inferior

performance of LoRA in knowledge- and skill-intensive domains comparing to full-fine tuning, such as code and math. Experimental study [206] demonstrates that the rank for full fine-tuning is significant (10-100×) higher than that for LoRA, and increasing the rank of LoRA updation can narrow the performance gap between LoRA and full fine-tuning. To increase the rank of LoRA and improve its performance, several methods have been proposed [17,24,27,209], which typically increase the rank through (1) stacking LoRAs along learning iterations; (2) updating as gradient compressors; (3) co-updating LLM and LoRA modules during fine-tuning.

3.1.1 Stacking LoRAs along fine-tuning

Matrix rank is subadditive, i.e., $rank(M_1+M_2)\leqslant rank(M_1)+rank(M_2)$ for matrices M_1 and M_2 that have the same size. Based on the subadditivity, we can aggregate multiple LoRA modules together to increase the rank and break the low-rank bottleneck. Following this idea, **ReLoRA** [17] proposes a merge-and-reinit procedure for LoRA, which periodically merges the LoRA modules to the LLM and then reinitializes the LoRA modules during fine-tuning. It equals stacking multiple LoRA modules along with fine-tuning and can increase the rank of the overall updates. Similarly, **COLA** [24] proposes another merge-and-reinit method based on Frank-Wolfe algorithm [210]. However, **MELoRA** [25] points out that the merge-and-reinit procedure does not necessarily guarantee an increase in rank, because there can be overlap between the series of LoRA modules along fine-tuning. To solve this problem, MELoRA proposes to decompose the LoRA modules into smaller mini LoRAs and then parallelly stack these mini LoRAs, whose effectiveness in increasing the rank is theoretically verified.

3.1.2 Updating as gradient compressor

The above methods break the low-rank bottleneck in the parameter space. As a supplement, **FLoRA** [26] finds that LoRA performs a fixed random projection to compress gradients and restricts the total weight matrix change to low-rank. To overcome this low-rank bottleneck in gradient space, **FLoRA** proposes to resample the random projection, which is demonstrated to largely recover the performance of full-matrix SGD.

3.1.3 Co-updating LLM and LoRA

The above two kinds of methods focus on improving the representation ability of LoRA itself. Different from them, **Delta-LoRA** [27] proposes to jointly update the LLM and LoRA modules, which directly updates the high-rank LLM and can gain better representation capable than updating LoRA independently. It updates the LLM based on the difference between two LoRA modules of two consecutive iterations, which enables it to update the LLM without any extra memory.

3.2 Dynamic rank allocation

For the rank of LoRA, higher is not always better. The abundant LoRA ranks may cause degeneration in both performance and efficiency. Furthermore, the importance of weights can vary across different layers of a Transformer model during fine-tuning, requiring different ranks for each layer [28,31,33,211]. Therefore, assigning the same rank to LoRA modules of different layers is not the optimal choice. It is better to adaptively allocate ranks to LoRA modules of different layers. Existing methods adaptively allocate ranks for LoRA modules from the perspectives of (1) singular value decomposition (SVD); (2) single-rank decomposition (SRD); (3) rank sampling.

3.2.1 SVD-based methods

Decomposing a matrix with singular value decomposition (SVD) and selectively truncating its singular values is an effective way to control the rank of the matrix. Inspire by SVD, we can decompose the LoRA parameter matrix BA into an SVD form, i.e., $P\Lambda Q$ where P and Q are orthogonal and Λ is a non-negative diagonal matrix. By controlling the elements in Λ, we can control the rank of BA and allocate ranks for LoRA modules. Following this idea, several rank allocation methods approximate the SVD decomposition for BA and allocate the ranks by filtering the diagonal matrix. For instance, **AdaLoRA** [28] approximates the SVD decomposition by regularizing the orthogonality of P and Q. Then, it drops unimportant singular values based on novel importance scoring methods. Similarly, **SaLoRA** [29] also introduces an orthogonality regularization for P and Q; by contrast, it drops unimportant singular values based on the L_0 norm. However, the above methods are not efficient enough for they start with a high rank and then reduce the rank iteratively, which brings a pre-defined budget [30]. To solve this problem, **IncreLoRA** [30] proposes to start from a single rank and then automatically increase the rank based on a heuristic importance score, where the orthogonality regularization is also involved while the elements in Λ is not required to be non-negative.

3.2.2 SRD-based methods

However, the orthogonality regularization brings unignorable computational costs for LoRA and degenerates its efficiency. To address this problem, several methods omit the orthogonality requirement of SVD and directly decompose BA into single-rank components. Then, they allocate the ranks by selecting the proper components. **DoRA (Dynamic Low-Rank Adaptation)** [31] proposes to decompose the LoRA parameter matrix BA into single-rank components and prunes the components based on a heuristic importance score. Similarly, **AutoLoRA** [32] also decomposes the LoRA parameter matrix BA into single-rank components, but it prunes the components based on meta-learning. **SoRA** [33] eliminates the orthogonality regularization and filters columns and rows of P and Q (their combination can be regarded as single-rank components) by directly controlling the diagonal matrix. It controls the diagonal matrix by formulating them as a set of learnable gating units which are updated in the fine-tuning procedure. **ALoRA** [34] also filters the components by using gating units; by contrast, it learns the gating units based on neural architecture search [212].

3.2.3 Rank sampling-based methods

In the SVD parameterization- and component-wise

decomposition-based methods, we need to spend the extra computational costs to search proper ranks. To avoid the extra cost, **DyLoRA** [35] points out that we can allocate ranks directly by random sampling. In each training step, it samples a value b from a pre-defined discrete distribution and allocates b as the rank. Then, the matrices A and B are truncated to rank-b. In the fine-tuning procedure, only the parameters on the bth row of A and bth column of B are tunable while other parameters are frozen. Besides, the distribution can be defined based on users' preferences.

3.3 Optimizing the learning procedure

In practice, LoRA converges more slowly than full fine-tuning. Moreover, it is also sensitive to hyperparameters and suffers from overfitting. These issues affect LoRA's efficiency and hinder its downstream adaptation performance. To address these issues, researchers have developed several approaches to optimize the learning procedure of LoRA, which can be categorized into the following three types: (1) Initialization Improvement; (2) Gradient Update Optimization; (3) Overfitting Mitigation.

3.3.1 Initialization improvement

LoRA usually initializes its parameter matrices A and B using Gaussian noise and zeros respectively. There are two simple schemes: Init[A], which sets matrix B to zero and randomly initializes matrix A, and Init[B], which does the reverse. Literature [36] compares these two schemes and concludes that Init[A] is better through theoretical analysis. It reveals that Init[A] allows using a larger learning rate without causing instability, making the learning process more efficient. However, even with Init[A], this random initialization method still results in small initial gradients, leading to slower convergence. To solve this, **PiSSA** [37] initializes LoRA with the principal singular components of the pre-trained matrix. Since principal singular components represent the most significant directions in the matrix, aligning the initial weights with these components can accelerate convergence and improve performance. In contrast, **MiLoRA** [38] initializes LoRA with the minor singular components. Given that random initialization of low-rank matrices can interfere with the important features learned in the pre-trained matrix, it reduces this interference to improve overall performance while adapting to new tasks.

3.3.2 Gradient update optimization

To further enhance the convergence and reliability of LoRA, several studies have proposed improvements from the perspective of gradient updates. [39] introduces a scaled gradient method based on Riemannian optimization, which incorporates an $r \times r$ preconditioner item in the gradient update step to improve the convergence and hyperparameter robustness of LoRA. Through theoretical analysis, **LoRA+** [40] discovered the necessity of setting a proportional learning rate for matrices A and B to achieve stable feature learning and accelerate convergence. **ResLoRA** [41] introduced residual connections into LoRA to optimize the gradient propagation path, speeding up training convergence and enhancing model performance. Similarly, **SIBO** [42] mitigate over-smoothing by injecting residual connections of initial token representations into LoRA's input. Additionally, to further reduce computational resources, literature [43] employs gradient-free optimization methods such as CMA-ES and FWA to optimize LoRA, demonstrating competitive performance in few-shot NLU tasks. Besides, **DoRA** (Weight-Decomposed Low-Rank Adaptation) [44] constrains the gradient update, focusing on the directional change of the parameter. It decomposes pre-trained weight into two components, direction and magnitude, and applies LoRA only to the direction component to enhance training stability.

3.3.3 Overfitting mitigation

Although LoRA effectively reduces the number of trainable parameters compared to full fine-tuning, some studies have shown that LoRA is also prone to overfitting [47], which contradicts previous views. To address this issue, **BiLoRA** [45] adopts a bi-level optimization strategy. It alternately trains the singular vectors and singular values of the low-rank increment matrix on different subsets of the training data. This approach avoids the simultaneous optimization of parameters at different levels on a single dataset, thus mitigating overfitting. In addition, literature [46] applies dropout to LoRA parameters to reduce overfitting, while **HiddenKey** [47] employs column-wise dropout for attention layers and element-wise dropout for feedforward layers.

3.4 Combining with other learning paradigms

LoRA is compatible with other learning paradigms, such as Bayesian Learning, In-context Learning and Active Learning. Combining LoRA with these learning paradigms can address several problems that hurt the downstream adaptation performance. For example, combining with Bayesian Learning, **Laplace-LoRA** [48] can relieve the overconfidence phenomenon that happened in downstream adaptation. Combining with In-context Learning, **PILLOW** [49] aims to solve the low-resource dilemmas existing in some downstream tasks. Combining with Active Learning, **STAR** [50] can effectively improve the data efficiency.

At last, to illustrate the performance difference between LoRA and some of its variants, we report their performance for RoBERTa-base [213] model on the GLUE benchmark [214] in Table 1. These results are derived from previous studies [9,16,32,45,90].

4 Cross-task generalization

LoRA's pluggable nature enables users to accumulate LoRA plugins for different tasks. For example, on Hugging Face platform, there are more than 20,000 LoRA plugins compatible with various LLMs for different tasks. These accumulated LoRA plugins can not only be utilized independently but also be mixed to achieve cross-task generalization [60]. Mixing multiple LoRA plugins together, namely LoRA mixture, has been widely applied in areas requiring cross-task generalization, such as multi-task learning, domain adaptation, and continual learning. Existing LoRA mixture methods can be categorized into (1) mixture with manually designed weights; (2) mixture with learnt weights; (3) mixture of LoRA experts. This section introduces

Table 1 Performance of LoRA and its variants for RoBERTa-base model on the GLUE benchmark. We report Matthew's correlation for CoLA, Pearson correlation for STS-B, and accuracy for the other datasets. The results are reported according to the results reported in literature [9,32,45,89,90]

Method	# Params	SST-2	MPRC	CoLA	QNLI	RTE	STS-B
Tied-LoRA [215]	0.043 M	94.4	88.5	61.9	92.0	76.2	89.8
AutoLoRA [32]	0.3 M	94.9	89.4	61.3	92.9	77.0	90.8
DyLoRA [35]	0.3 M	94.3	89.5	61.1	92.2	78.7	91.1
AdaLoRA [28]	0.3 M	94.5	88.7	62.0	93.1	81.0	90.5
FourierFT [90]	0.024 M	94.2	90.0	63.8	92.2	79.1	90.8
VeRA [88]	0.043 M	94.6	89.5	**65.6**	91.8	78.7	90.7
Full Fine-tuning [9]	125 M	94.8	90.2	63.6	92.8	78.7	91.2
LoRA [9]	0.3 M	**95.1**	89.7	63.4	**93.3**	78.4	91.5
VB-LoRA [89]	0.023 M	94.4	89.5	63.3	92.2	82.3	90.8
BiLoRA [45]	0.3 M	**95.1**	**91.7**	64.8	**93.3**	**87.2**	**91.7**

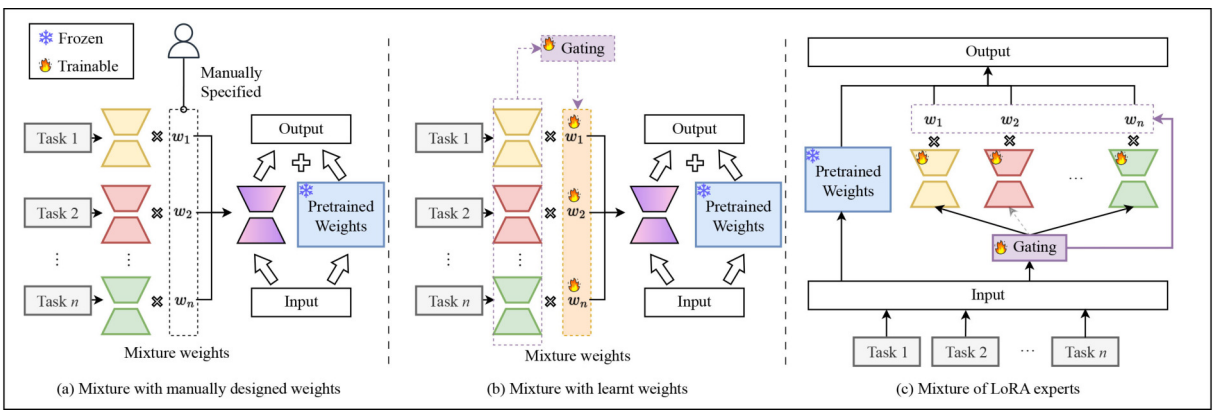


Fig. 3 An illustration of LoRA mixture methods

each category of methods respectively, as shown in Fig. 3.

4.1 Mixture with manually designed weights

Early LoRA mixture methods attempt to linearly combine different LoRA modules with manually designed weights. Some research demonstrates that we can achieve proper cross-task generalization ability by simply averaging LoRA modules or their related outputs [51–53]. Furthermore, several methods have been proposed to further improve the performance of the LoRA mixture via adopting manually designed weights. For example, **ControlPE** [54], [55] and [56] set the weight factors as hyperparameters, and ControlPE uses hyperparameter search to determine the optimal combination of two LoRA modules. Additionally, **Token-Level Adaptation** [57] utilizes cosine similarity between the input feature and the adapter dataset center as weight factors, while **BYOM** [58] applies basic model fusion methods such as Task Arithmetic, Fisher-Merging, and RegMean.

Mixture with manually designed weights can quickly mix multiple LoRAs without extra training, which demonstrates simplicity and computational efficiency. However, it often fails to find the optimal weights, leading to unstable performance and limited generalization. Subsequently, researchers have explored using learning-based methods to achieve more precise and adaptive mixtures.

4.2 Mixture with learnt weights

To learn the optimal mixture weights, several methods have been proposed at task level, instance level and token level to meet different needs. Task-level methods focus on enhancing task transferability, which can be either gradient-based, such as [59], or gradient-free, as seen in **LoRAHub** [60]. LoRAHub employs a black-box algorithm named CMA-ES [216] to optimize weight factors for LoRA modules, simplifying the training process. Later, **ComPEFT** [61] and **L-LoRA** [62] use LoRAHub to mix quantized LoRA modules, further improving computational efficiency.

Compared to task-level methods, instance-level and token-level methods can provide flexibility and precision for complex inputs. For multimodal instruction tuning, **MixLoRA** [63] dynamically chooses appropriate low-rank decomposition vectors based on the input instance, which are then integrated into LoRA matrices for training. To conduct protein mechanics analysis and design tasks, **X-LoRA** [64] develops a dynamic gating mechanism to assign weights for LoRA modules at the token level and layer granularity. These approaches demonstrate better performance in specific tasks or application scenarios.

4.3 Mixture of LoRA experts

When the LoRA modules are trainable, we can jointly learn the mixture weights and the LoRA modules, which can further improve the performance of the LoRA mixture. To jointly learn the mixture weights and LoRA modules, Mixture of LoRA Experts (LoRA MoE) is a natural choice, where each LoRA module acts as an expert, while a router network typically assigns the mixture weights. LoRA MoE has been proven to be effective in many tasks, such as continual learning [65,66], vision-language tasks [67] and multi-task medical applications [68].

Existing methods improve the performance of LoRA MoE from the perspectives of initialization, task relationship management and efficiency. For initialization, **Mixture-of-LoRAs** [69] first trains multiple LoRAs separately as initialization and then optimizes the router and LoRAs jointly. **MultiLoRA** [70] proposes refining the initialization to reduce parameter dependency, which can yield more balanced unitary subspaces. As for task balance, **MLoRE** [71] adds a low-rank convolution path in the MoE structure to capture global task relationships. **MTLoRA** [72] adopts both task-agnostic and task-specific LoRA modules to address task conflicts. For efficiency, **MoLA** [73] adaptively allocates different numbers of LoRA experts to different layers of the Transformer model to save the number of LoRA modules. **LLaVA-MoLE** [74] and **SiRA** [75] leverage sparse computation to reduce computational cost. Additionally, **Octavius** [76] sparsely activates independent LoRA experts with instance-level instructions to mitigate task interference and improve efficiency. **Fast LoRA** [77] allows each sample in a minibatch to have its unique low-rank adapters, enabling efficient batching.

Besides, some methods are not explicitly based on MoE but follow MoE ideas. For example, **MoSLoRA** [78] decomposes LoRA into subspaces and employs a learnable mixer to fuse these subspaces.

5 Efficiency improving

With the popularization of LLMs, the demand for training and running LoRA modules increases rapidly. This increasing demand brings an unignorable computational burden; thus, for LoRA, the smaller, the faster, the better. To meet this demand, existing methods improve the computational efficiency of LoRA from the perspectives of (1) parameter reduction; (2) parameter quantization; (3) parallel LoRA computing frameworks. This section introduces each category of methods, as illustrated in Fig. 4.

5.1 Parameter reduction

LoRA significantly reduces the number of tunable parameters for fine-tuning LLMs. However, it still requires expensive activation memory to update low-rank matrices. To further reduce the memory cost, existing methods reduce the number of tunable parameters of LoRA via parameter freezing, parameter pruning, and parameter sharing.

5.1.1 Parameter freezing

Parameter freezing methods reduce the number of tunable parameters for LoRA via freezing some of its parameters. They can be divided into two categories: intra-parameter methods and extra-parameter methods.

The intra-parameter methods tune a subset of parameters of LoRA while freezing the others. **LoRA-SP** [79] randomly selects half of the LoRA parameters to freeze during fine-tuning. **LoRA-FA** [80]freezes the down-projection weights and updates the up-projection weights in each layer of LoRA. **AFLoRA** [81] constructs a low-rank trainable path and gradually freezes parameters during training LoRA. Additionally, **DropBP** [82] accelerates the training process by randomly dropping some LoRA gradient calculations during backpropagation.

By contrast, the extra-parameter methods introduce and tune a set of extra parameters while freezing the original parameters of LoRA. Most of them are proposed based on Singular Value Decomposition (SVD). **LoRA-XS** [83] adds a small $r \times r$ weight matrix between frozen LoRA matrices, which are constructed using the SVD of the original weight matrix; then it tunes only the $r \times r$ weight matrices in fine-tuning. Similarly, **BYOM-LoRA** [58] adopts SVD to compress LoRA matrices for multi-task models.

5.1.2 Parameter pruning

Parameter pruning methods aim to remove unimportant LoRA parameters during training and inference. They prune parameters by either pruning LoRA independently or jointly pruning LoRA and the LLM. **LoRA-drop** [84] uses the output of LoRA at each layer to evaluate the importance of parameters and prune the unimportant parameters. By contrast, **LoRAPrune** [85] jointly pruning LoRA matrices and the LLM parameters based on LoRA's gradients. Besides, we can also use LoRA to support parameters pruning for LLMs [86,87].

5.1.3 Parameter sharing

Parameter-sharing methods reduce the number of parameters by sharing parameters across different layers or modules of LLMs. **VeRA** [88] and **VB-LoRA** [89] are two representative parameter-sharing methods for LoRA. Specifically, VeRA proposes to share a pair of frozen random matrices across all layers and conduct layer-wise adaptation with "scaling vectors". By contrast, VB-LoRA proposes a "divide-and-

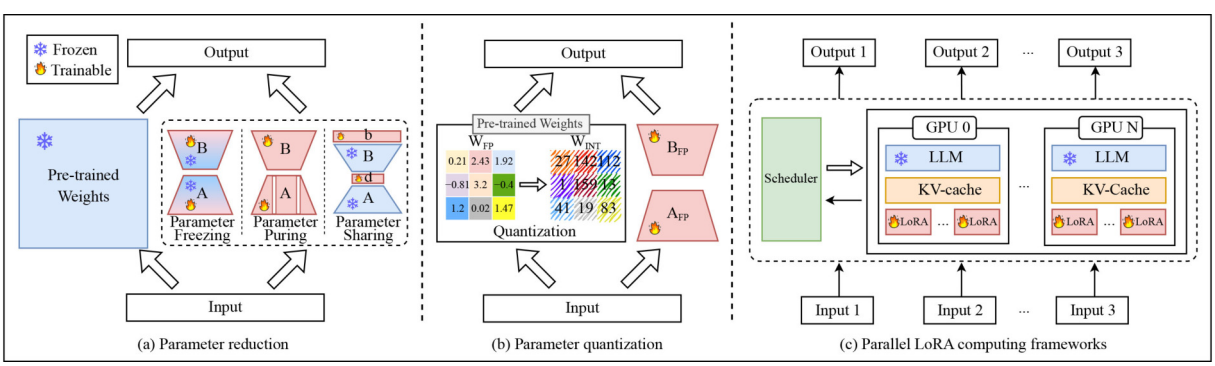


Fig. 4 An illustration of efficiency improving methods

share" paradigm, which divides LoRA's low-rank decomposition by a rank-one decomposition and achieves global sharing based on an admixture model. Instead of sharing parameters in the original parameter space, **FourierFT** [90] converts the incremental matrix ΔW into the spatial domain using Fourier transform. It shares spectral entries across all layers and only learns its sparse spectral coefficients for each layer, thus reducing the number of trainable parameters.

5.2 Parameter quantization

Quantization, which reduces the bit width of parameters (e.g., from 32-bit floats to 4-bit integers), can be used to reduce the memory and computational cost of LoRA. Existing quantization-aware LoRA methods consist of post-training quantization (PTQ)-based methods and quantization-aware training (QAT)-based methods [95].

5.2.1 PTQ-based methods

In PTQ-based methods, we first quantize an LLM and then fine-tune the quantized model, namely quantization and fine-tuning are sequentially conducted. **QLoRA** [91] is the first PTQ-based quantization-aware LoRA method. In the fine-tuning stage, it first quantizes an LLM to 4 bits and then fine-tunes a LoRA module on it with a higher precision, such as BFloat16 or Float16. In the inference stage, it dequantizes the LLM to the same precision as LoRA and then adds the LoRA updates to the LLM.

Although QLoRA can significantly reduce memory cost for fine-tuning, it does not bring benefits for inference, because it requires dequantizing the LLM to high precision again. To solve this problem, **QA-LoRA** [92] is proposed to reduce memory cost for both the fine-tuning and inference stages. QA-LoRA uses group-wise operators to balance the degrees of freedom of the LLM quantization and fine-tuning, which enables it to obtain a LoRA module having identical precision with the quantized LLM. Thus, it can perform inference without dequantization.

5.2.2 QAT-based methods

In QAT-based methods, we jointly quantize and fine-tune an LLM, namely quantization and fine-tuning are simultaneously conducted. These methods can alleviate the quantization discrepancies observed in PTQ-based methods. To address the quantization discrepancy of QLoRA, **LoftQ** [93] alternatively applies quantization and low-rank approximation during fine-tuning to minimize the quantization error. However, **ApiQ** [94] points out that LoftQ ignores the error propagation across layers and proposes activation-preserved initialization to avoid error propagation. Besides, **L4Q** [95] is another QAT-based method that has an advanced layer design.

5.3 Parallel LoRA computing frameworks

LoRA's parameter-efficient nature enables us to fine-tune or infer multiple modules on a single GPU or a GPU cluster, which can save computational resources and improve the efficiency of LoRA. This section introduces the parallel fine-tuning and parallel inference frameworks, respectively.

5.3.1 Parallel fine-tuning

Parallelly fine-tuning multiple LoRA modules on a single GPU can reduce GPU memory usage and improve computation efficiency. **ASPEN** [96] proposes a high-throughput parallel finetuning framework for LoRA, which consists of a BatchFusion approach and an adaptive job scheduling algorithm. Specifically, the BatchFusion approach supports parallelly fine-tuning multiple LoRA modules on a shared LLM by fusing multiple input batches into a single batch, while the adaptive job scheduling algorithm allocates computation resources to the fine-tuning jobs.

5.3.2 Parallel inference

Parallel inference framework for LoRA can not only improve the computational efficiency but also support the needs of multi-tenant service. **Punica** [97] uses a new CUDA kernel design to batch GPU operations for different LoRA modules. Based on Punica, **S-LoRA** [98] further optimizes the parallel inference framework by introducing a unified paging mechanism and a new tensor parallelism strategy, which enables the service of thousands of concurrent LoRA modules. Then, based on Punica and S-LoRA, **CaraServe** [99] reduces the cold-start overhead and further improves the service efficiency and SLO (service-level objective) attainment rates by CPU-GPU cooperation and rank-aware scheduling.

6 LoRA for federated learning

When adapting LLMs to vertical domains such as medicine and finance, the available training data can be privately owned by multiple clients. In this scenario, the training data is not centralized, and we have to fine-tune LLMs while keeping the data localized, namely federated learning. In federated learning, the clients typically compute weight updates locally and then share these updates with others to globally update the

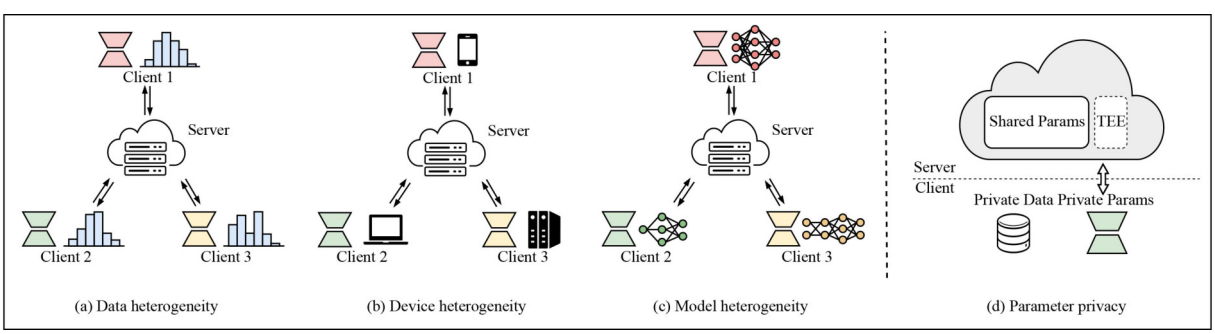


Fig. 5 An illustration of LoRA for federated learning

LLM. It brings both communication and computation costs for the clients. Fortunately, LoRA is parameter efficient and pluggable, which can reduce communication costs and lower computational resource requirements. LoRA can enhance the overall efficiency and scalability of federated learning.

However, adopting LoRA in federated learning is not trivial for federated learning faces challenges such as data heterogeneity, device heterogeneity, and model heterogeneity. To address these issues, recent studies have designed various methods for LoRA to meet the diverse needs of federated learning, as shown in Fig. 5. Additionally, as a localized parameter component, LoRA's pluggable nature allows it to support parameter privacy protection in federated learning.

6.1 Data heterogeneity

Data heterogeneity refers to differences in data distribution across clients. In federated learning, different clients usually have different data distributions. The inconsistency in data distribution affects the overall performance of the model. Research reveals that in federated learning, as user data becomes more diverse, the performance gap between LoRA and full fine-tuning widens [100]. To address this issue, researchers have proposed several improvement methods.

SLoRA [100] introduces a data-driven initialization method for LoRA. It first performs sparse federated fine-tuning before applying LoRA and then performs SVD to decompose the accumulated gradient updates into low-rank matrices for LoRA initialization. The goal is to enable the LoRA modules to better adapt to the data distribution of each client, thereby integrating these heterogeneous data characteristics into the global model more effectively. **FeDeRA** [101] uses a simpler initialization method. It directly applies SVD to pre-trained weights to initialize LoRA. Retaining the principal components of the pre-trained weights aligns the direction and magnitude of weight updates across different clients to handle data heterogeneity. Additionally, **FFA-LoRA** [102] freezes one low-rank matrix and fine-tunes only the other. This reduces inconsistency during server aggregation of LoRA gradients, alleviating the optimization instability caused by non-IID data.

6.2 Device heterogeneity

Device heterogeneity refers to the differences in hardware capabilities, and network connectivity among clients participating in federated learning. Traditional federated learning methods often encounter the "buckets effect", implying that the system's overall performance is limited by the capability of the least powerful client. Specifically, these methods use the smallest LoRA rank to accommodate all clients, which prevents many resource-rich clients from fully utilizing their potential.

To address this issue, a dynamic parameter allocation strategy can be adopted. **FedMS** [103] dynamically adjusts the number of activated LoRA matrices based on the real-time computational resources of clients. **FlexLoRA** [104] uses a dynamic parameter allocation strategy. It adjusts the LoRA rank and redistributes the SVD components of the global LoRA weights based on resource constraints. Similarly, **HETLORA** [105] assigns different ranks for different clients. However, it performs weighted aggregation according to the sparsity of the updates from different clients, balancing update information better than simple aggregation.

6.3 Model heterogeneity

Model heterogeneity indicates differences in model structures among clients. In traditional federated learning, clients use local models with the same architecture, allowing their parameters to be aggregated into a global model on the server. However, in practice, clients may prefer unique local model architectures due to personal needs and often do not want to disclose model details. Thus, it is necessary to transfer knowledge between heterogeneous models without sharing private data or revealing local model structures [217].

Previous work has used knowledge distillation, model ensembling, and mutual learning to address model heterogeneity. However, these methods have limitations, such as reliance on public datasets, additional communication costs and poor local model performance. To avoid these limitations, **pFedLoRA** [106] uses LoRA as a carrier of both global and local knowledge. It adopts an iterative training strategy to facilitate knowledge transfer and integration, enabling knowledge sharing among heterogeneous models across different clients.

6.4 Parameter privacy

In federated learning, protecting client-specific parameters is crucial because ensuring the privacy of these parameters also indirectly safeguards client data privacy. As a modular approach to adjusting personalized parameters, LoRA can be effectively integrated into federated learning systems to achieve parameter privacy protection.

Literature [107] proposes a secure distributed language model training framework based on model slicing. They deploy LoRA in a Trusted Execution Environment (TEE) and use OTP encryption to transmit features between the GPU and TEE, protecting model parameter privacy. **PrivateLoRA** [108] introduces a distributed system based on LoRA. It adds a square matrix M between low-rank matrices A and B. The non-trainable matrices A and B, along with most of the pre-trained weights, are deployed on the global server to enhance computation. Meanwhile, the trainable matrix M is stored on the client as personalized parameters, thus ensuring parameter privacy protection.

Furthermore, recent works have integrated differential privacy (DP) techniques with LoRA in federated learning to enhance data privacy. **DP-LoRA** [218] ensures differential privacy by adding Gaussian noise to LoRA's weight updates during the update process. This approach maintains privacy and improves communication efficiency. To solve the noise amplification when applying differential privacy in LoRA, **FFA-LoRA** [102] fixes the matrix A, avoiding the local semi-quadratic structure and enhancing robustness and performance.

7 Applications of LoRA

In the rapidly evolving field of deep learning, LoRA has become widely used due to its unique advantages. Researchers

utilize LoRA to fine-tune pre-trained models for various downstream tasks, reducing computational resource requirements while enhancing performance. LoRA's strong adaptability and efficiency have significantly improved various applications. In this section, we will introduce LoRA's applications in the following scenarios: (1) language tasks; (2) vision tasks; (3) multimodal tasks.

7.1 Language tasks

Recently, the rapid development of pre-trained language models, especially LLMs, is revolutionizing the approach to language tasks due to their outstanding performance. However, these pre-trained models are trained on a large amount of general data and still require further fine-tuning on task-specific data to adapt to downstream tasks. Therefore, it is natural to use LoRA to fine-tune these pre-trained language models, as it reduces computational resource requirements. We mainly focus on some representative downstream tasks, which include traditional NLP tasks, code tasks, model alignment and vertical domain tasks.

7.1.1 Traditional NLP tasks

Given the strong instruction-following and contextual understanding abilities of LLMs, some researches apply LoRA to fine-tune these models for traditional NLP tasks. For example, LoRA is widely adopted in LLaMA for various tasks, such as emotion recognition [109], text classification [110] and role recognition [111]. **AutoRE** [112] applies QLoRA to three document-level relation extraction tasks, achieving great performance on different LLMs. Some studies [113–115] leverage LoRA from different perspectives to enhance the model's capability in machine translation tasks. Additionally, LoRA can also improve the performance of models like BERT and T5 for text understanding tasks [116,117].

7.1.2 Code tasks

Some researchs apply LoRA to improve model performance in various code-related tasks. For example, BERT-style models fine-tuned with LoRA are suitable for code-change-related tasks, specifically in Just-In-Time defect prediction (JIT-DP) [118,119]. Similarly, training CodeT5 and PLBART with LoRA can enhance their adaptability for code summarization and code clone detection [120]. As for the decoder-only model, **RepairLLaMA** [121] uses LoRA to fine-tune Llama for automated program repair (APR), while WizardCoder-15B is fine-tuned with LoRA for Text-to-SQL task [122]. Additionally, **SteloCoder** [123], a fine-tuned version of StarCoder, is designed for multi-language to Python code translation.

7.1.3 Model alignment tasks

Model alignment tasks focus on adjusting a machine learning model to align with human values and intentions, often using techniques like Reinforcement Learning from Human Feedback (RLHF). To reduce memory requirements of RLHF, some studies use LoRA to fine-tune the reward model and policy model [124–126]. Furthermore, other works improve reward models by integrating multiple LoRA adapters. For example, **DMoERM** [127] combines MoE with LoRA, routing model inputs to multiple LoRA experts while another work [128] proposes a LoRA-based ensemble method as well. The integration can also benefit the quantification of uncertainty in reward models [129]. Besides, literature [130] applies Laplace-LoRA with a Gaussian prior assumption [131] to train Bayesian reward models, which mitigates reward overoptimization in best-of-n sampling.

7.1.4 Vertical domain tasks

LLMs often perform suboptimally in vertical domains, requiring fine-tuning with domain-specific expertise. Some works apply LoRA to improve the performance of LLMs on domain-specific tasks. For example, some studies fine-tune LLMs on medical datasets with LoRA to adapt them to the medical domain [132–134]. Additionally, other studies improve medical tasks like clinical dialogue summarization [135], assertion detection [136] and medical QA tasks [137,138]. Similarly, several studies fine-tune LLMs with LoRA on financial data to solve tasks such as financial news analytics and sentiment classification [139–142]. Besides, LoRA can also be used to enhance the performance in database tasks like query rewrite and index tuning [143].

7.2 Vision tasks

In vision tasks, LoRA is primarily applied to image generation and image segmentation, significantly improving training efficiency and optimizing model performance.

7.2.1 Image generation

Image generation tasks hold significant importance in the field of computer vision. In recent years, diffusion model have demonstrated exceptional performance in image generation tasks. LoRA is widely used in diffusion models to address various image generation tasks while reducing computational resources. Some works use LoRA to fine-tune diffusion models for image style transfer [144–148], while others apply it to text-to-image generation [149–153].

Furthermore, researchers have designed several LoRA-based methods to improve image generation quality. For instance, **Smooth Diffusion** [154] uses LoRA to achieve smoothness in the latent space, leading to better performance in various image generation and editing tasks. **ResAdapter** [155] employs LoRA to learn resolution priors, adjusting the receptive fields of convolutional layers to dynamical resolution. Additionally, to specifically enhance text-to-image quality, **STAMINA** [156] uses LoRA to fine-tune diffusion models for longer concept sequences. **DreamSync** [157] and **StyleAdapter** [158] use LoRA to improve text fidelity and image quality. **Mix-of-Show** [159] captures out-of-domain information with LoRA weights to combine multiple customized concepts with high fidelity, reducing concept conflicts. Other studies combine LoRA with model distillation to accelerate image generation [160,161]. Moreover, LoRA can also be applied to video generation [162–167] and 3D generation tasks [168–172].

7.2.2 Image segmentation

Image segmentation is a significant challenge in computer

vision, aiming to divide an image into multiple meaningful regions or objects. To address this, SAM has been proposed as a foundational model for image segmentation and demonstrated superior generalization ability. To further enhance its performance in specific vertical domains, many studies utilize LoRA to fine-tune it. For instance, in license plate detection, **SamLP** [173] utilizes LoRA to adapt SAM for efficient segmentation of license plates. In structural damage detection, literature [174] fine-tunes SAM's encoder using LoRA for instance segmentation task. In the medical domain, many studies also apply LoRA to fine-tune SAM for a variety of tasks, including nuclei segmentation [175], OCTA image segmentation [176], brain tumor segmentation [177], organ segmentation [178], and surgical instrument segmentation [179]. Additionally, some studies use LoRA to fine-tune Vision Transformer (ViT) for visual tracking [180] and face forgery detection [181].

7.3 Multimodal tasks

Multimodal Large Language Models (MLLMs) aim to integrate text with various modalities such as audio, image and video, which enable cross-modal understanding and reasoning through a unified embedding space. The success of LoRA in both NLP and vision tasks has sparked considerable interest in applying them to MLLMs.

In MLLMs, LoRA can not only improve training efficiency but also facilitate effective modality alignment. In audio-text tasks, **SALM** [182] comprises LoRA layers, a frozen text-based LLM, an audio encoder and a modality adapter to handle speech inputs and corresponding task instructions. For image-text tasks, **InternLM-XComposer2** [183] achieves modality alignment by applying LoRA to image tokens, **mPLUG-Owl** [184] freezes the visual module while jointly fine-tuning LoRA and abstractor of the text module, and **CoLLaVO** [185] employs QLoRA to preserve object-level image understanding. In the realm of video-text tasks, **VSP-LLM** [186] fine-tunes the text module with QLoRA for visual speech processing, **MolCA** [187] uses LoRA to understand 2D molecular graphs and text, while **TPLLM** [188] employs LoRA for efficient traffic prediction by integrating sequence and spatial features. These applications demonstrate the versatility and power of LoRA in MLLMs tasks.

8 Conclusion and future direction

In this survey, the recent progress of LoRA have been systematically reviewed from the perspective of downstream adaptation improving, cross-task generalization, efficiency improving, federated learning and applications. From this review, we can find that LoRA is parameter efficient, pluggable, compatible and easy to achieve cross-task generalization, which enables it to be one of the most important technology for LLMs applications. Recent progress further boosts the generalization and efficiency of LoRA, and stimulate its potential to be used in more scenarios. Here, we list three future directions where LoRA will be indispensable.

8.1 LoRA for GaaS

In Generative-as-a-Service (GaaS), cloud-based platforms provide users with generative artificial intelligence (AGI) services. GaaS enables users enjoy AGI without deploying local computational resources. For the users' needs are diverse, it is necessary to provides various functions for GaaS. To implement the various functions, we can construct a LoRA module for each function. The pramameter efficiency and plugability of LoRA can facilitate efficient functions' construction and execution. Besides, the services on GaaS platforms can change rapidly alonging time. To follow the changes, we can train new LoRA modules that initialized by combination of previous LoRA modules. The cross-task generalization ability of LoRA can facilitate fast adaption to service updations.

8.2 LoRA for continued pre-training

In continued pre-training, a foundation model is continuely trained with unlabeled user data to adapt the model to specific domains. Typically, the self-supervised training objective is same with that for pre-training, and the learning rate is much smaller than than for pre-training. Continued pre-training is a important stage for constructing vertical domain LLMs. However, it is highly computational expensive, which impedes the development of vertical domain LLMs, especailly for the organizations with limited computational resources. Enhancing LoRA for continued pre-training and reducing its computational cost is worth to explored.

8.3 LoRA for autonomous agents

In LLM-based autonomous agents, the agents are assigned with specific roles. Based the roles and environment, agents make actions to response users' or other agents' request. The actions can be made based on self-knowledge or tools that designed for domain-specific tasks. The request and the actions are stored in memory to support the future requests.

In the current agents, the roles are typically assigned by prompts; however, prompt may cannot give a comprehensive discription of the role when the role is complex and the number of related data is large. Assiging roles with LoRA modules training from data related to the roles can be a better choice. Furthermore, the tools for agent can be LoRA modules. Besides, the memory usually augments the agents with retrieval augmented generation (RAG); however, due to the input token limitation and the short-comings of in-context learning, the RAG-based support may be less effective. By contrast, we can use LoRA-based continual learning to construct memory modules, which can solve the problem of RAG. Therefore, LoRA-driven agents are worth to explore.

Acknowledgements This work was supported in part by the National Natural Science Foundation of Chian (Grant Nos. 62025206, 62302436, U23A20296), the Zhejiang Province's "Lingyan" R&D Project (No. 2024C01259), and the Ningbo Science and Technology Special Projects (No. 2023Z212).

Competing interests The authors declare that they have no competing interests or financial conflicts to disclose.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made.

The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Devlin J, Chang M W, Lee K, Toutanova K. BERT: pre-training of deep bidirectional transformers for language understanding. In: Proceedings of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL-HLT. 2019, 4171−4186
2. Chowdhery A, Narang S, Devlin J, Bosma M, Mishra G, Roberts A, Barham P, Chung H W, Sutton C, Gehrmann S, Schuh P, Shi K, Tsvyashchenko S, Maynez J, Rao A, Barnes P, Tay Y, Shazeer N, Prabhakaran V, Reif E, Du N, Hutchinson B, Pope R, Bradbury J, Austin J, Isard M, Gur-Ari G, Yin P, Duke T, Levskaya A, Ghemawat S, Dev S, Michalewski H, Garcia X, Misra V, Robinson K, Fedus L, Zhou D, Ippolito D, Luan D, Lim H, Zoph B, Spiridonov A, Sepassi R, Dohan D, Agrawal S, Omernick M, Dai A M, Pillai T S, Pellat M, Lewkowycz A, Moreira E, Child R, Polozov O, Lee K, Zhou Z, Wang X, Saeta B, Diaz M, Firat O, Catasta M, Wei J, Meier-Hellstern K, Eck D, Dean J, Petrov S, Fiedel N. PaLM: scaling language modeling with pathways. The Journal of Machine Learning Research, 2023, 24(1): 240
3. Chen Y, Qian S, Tang H, Lai X, Liu Z, Han S, Jia J. LongLoRA: efficient fine-tuning of long-context large language models. In: Proceedings of the 12th International Conference on Learning Representations. 2024
4. Pan R, Liu X, Diao S, Pi R, Zhang J, Han C, Zhang T. LISA: layerwise importance sampling for memory-efficient large language model fine-tuning. 2024, arXiv preprint arXiv: 2403.17919
5. Ding N, Qin Y, Yang G, Wei F, Yang Z, Su Y, Hu S, Chen Y, Chan C M, Chen W, Yi J, Zhao W, Wang X, Liu Z, Zheng H T, Chen J, Liu Y, Tang J, Li J, Sun M. Parameter-efficient fine-tuning of large-scale pre-trained language models. Nature Machine Intelligence, 2023, 5(3): 220−235
6. Houlsby N, Giurgiu A, Jastrzebski S, Morrone B, de Laroussilhe Q, Gesmundo A, Attariyan M, Gelly S. Parameter-efficient transfer learning for NLP. In: Proceedings of the 36th International Conference on Machine Learning. 2019, 2790−2799
7. Lester B, Al-Rfou R, Constant N. The power of scale for parameter-efficient prompt tuning. In: Proceedings of 2021 Conference on Empirical Methods in Natural Language Processing. 2021, 3045−3059
8. Zaken E B, Goldberg Y, Ravfogel S. BitFit: simple parameter-efficient fine-tuning for transformer-based masked language-models. In: Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers). 2022, 1−9
9. Hu E J, Shen Y, Wallis P, Allen-Zhu Z, Li Y, Wang S, Wang L, Chen W. LoRA: low-rank adaptation of large language models. In: Proceedings of the 10th International Conference on Learning Representations. 2022
10. Zhao W X, Zhou K, Li J, Tang T, Wang X, Hou Y, Min Y, Zhang B, Zhang J, Dong Z, Du Y, Yang C, Chen Y, Chen Z, Jiang J, Ren R, Li Y, Tang X, Liu Z, Liu P, Nie J Y, Wen J R. A survey of large language models. 2023, arXiv preprint arXiv: 2303.18223
11. Han Z, Gao C, Liu J, Zhang J, Zhang S Q. Parameter-efficient fine-tuning for large models: a comprehensive survey. 2024, arXiv preprint arXiv: 2403.14608
12. Malladi S, Wettig A, Yu D, Chen D, Arora S. A kernel-based view of language model fine-tuning. In: Proceedings of the 40th International Conference on Machine Learning. 2023, 23610−23641
13. Koubbi H, Boussard M, Hernandez L. The impact of LoRA on the emergence of clusters in transformers. 2024, arXiv preprint arXiv: 2402.15415
14. Jang U, Lee J D, Ryu E K. LoRA training in the NTK regime has no spurious local minima. 2024, arXiv preprint arXiv: 2402.11867
15. Zhu J, Greenewald K, Nadjahi K, de Ocáriz Borde H S, Gabrielsson R B, Choshen L, Ghassemi M, Yurochkin M, Solomon J. Asymmetry in low-rank adapters of foundation models. 2024, arXiv preprint arXiv: 2402.16842
16. Zeng Y, Lee K. The expressive power of low-rank adaptation. In: Proceedings of the 12th International Conference on Learning Representations. 2024
17. Lialin V, Muckatira S, Shivagunde N, Rumshisky A. ReLoRA: high-rank training through low-rank updates. In: Proceedings of the 12th International Conference on Learning Representations. 2024
18. Jiang T, Huang S, Luo S, Zhang Z, Huang H, Wei F, Deng W, Sun F, Zhang Q, Wang D, Zhuang F. MoRA: high-rank updating for parameter-efficient fine-tuning. 2024, arXiv preprint arXiv: 2405.12130
19. Huh M, Cheung B, Bernstein J, Isola P, Agrawal P. Training neural networks from scratch with parallel low-rank adapters. 2024, arXiv preprint arXiv: 2402.16828
20. Liang Y S, Li W J. InfLoRA: interference-free low-rank adaptation for continual learning. 2024, arXiv preprint arXiv: 2404.00228
21. Zhao H, Ni B, Wang H, Fan J, Zhu F, Wang Y, Chen Y, Meng G, Zhang Z. Continual forgetting for pre-trained vision models. 2024, arXiv preprint arXiv: 2403.11530
22. Ren W, Li X, Wang L, Zhao T, Qin W. Analyzing and reducing catastrophic forgetting in parameter efficient tuning. 2024, arXiv preprint arXiv: 2402.18865
23. Zhang H. SinkLoRA: enhanced efficiency and chat capabilities for long-context large language models. 2024, arXiv preprint arXiv: 2406.05678
24. Xia W, Qin C, Hazan E. Chain of LoRA: efficient fine-tuning of language models via residual learning. 2024, arXiv preprint arXiv: 2401.04151
25. Ren P, Shi C, Wu S, Zhang M, Ren Z, de Rijke M, Chen Z, Pei J. MELoRA: mini-ensemble low-rank adapters for parameter-efficient fine-tuning. 2024, arXiv preprint arXiv: 2402.17263
26. Hao Y, Cao Y, Mou L. Flora: low-rank adapters are secretly gradient compressors. 2024, arXiv preprint arXiv: 2402.03293
27. Zi B, Qi X, Wang L, Wang J, Wong K F, Zhang L. Delta-LoRA: fine-tuning high-rank parameters with the delta of low-rank matrices. 2023, arXiv preprint arXiv: 2309.02411
28. Zhang Q, Chen M, Bukharin A, He P, Cheng Y, Chen W, Zhao T. Adaptive budget allocation for parameter-efficient fine-tuning. In: Proceedings of the 11th International Conference on Learning Representations. 2023
29. Hu Y, Xie Y, Wang T, Chen M, Pan Z. Structure-aware low-rank adaptation for parameter-efficient fine-tuning. Mathematics, 2023, 11(20): 4317
30. Zhang F, Li L, Chen J, Jiang Z, Wang B, Qian Y. IncreLoRA: incremental parameter allocation method for parameter-efficient fine-tuning. 2023, arXiv preprint arXiv: 2308.12043
31. Mao Y, Huang K, Guan C, Bao G, Mo F, Xu J. DoRA: enhancing parameter-efficient fine-tuning with dynamic rank distribution. 2024, arXiv preprint arXiv: 2405.17357

32. Zhang R, Qiang R, Somayajula S A, Xie P. AutoLoRA: automatically tuning matrix ranks in low-rank adaptation based on meta learning. 2024, arXiv preprint arXiv: 2403.09113
33. Ding N, Lv X, Wang Q, Chen Y, Zhou B, Liu Z, Sun M. Sparse low-rank adaptation of pre-trained language models. In: Proceedings of 2023 Conference on Empirical Methods in Natural Language Processing. 2023, 4133−4145
34. Liu Z, Lyn J, Zhu W, Tian X, Graham Y. ALoRA: allocating low-rank adaptation for fine-tuning large language models. 2024, arXiv preprint arXiv: 2403.16187
35. Valipour M, Rezagholizadeh M, Kobyzev I, Ghodsi A. DyLoRA: parameter-efficient tuning of pre-trained models using dynamic search-free low-rank adaptation. In: Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics. 2023, 3274−3287
36. Hayou S, Ghosh N, Yu B. The impact of initialization on LoRA finetuning dynamics. 2024, arXiv preprint arXiv: 2406.08447
37. Meng F, Wang Z, Zhang M. PiSSA: principal singular values and singular vectors adaptation of large language models. 2024, arXiv preprint arXiv: 2404.02948
38. Wang H, Xiao Z, Li Y, Wang S, Chen G, Chen Y. MiLoRA: harnessing minor singular components for parameter-efficient LLM finetuning. 2024, arXiv preprint arXiv: 2406.09044
39. Zhang F, Pilanci M. Riemannian preconditioned LoRA for fine-tuning foundation models. 2024, arXiv preprint arXiv: 2402.02347
40. Hayou S, Ghosh N, Yu B. LoRA+: efficient low rank adaptation of large models. 2024, arXiv preprint arXiv: 2402.12354
41. Shi S, Huang S, Song M, Li Z, Zhang Z, Huang H, Wei F, Deng W, Sun F, Zhang Q. ResLoRA: identity residual mapping in low-rank adaption. 2024, arXiv preprint arXiv: 2402.18039
42. Wen Z, Zhang J, Fang Y. SIBO: a simple booster for parameter-efficient fine-tuning. 2024, arXiv preprint arXiv: 2402.11896
43. Jin F, Liu Y, Tan Y. Derivative-free optimization for low-rank adaptation in large language models. 2024, arXiv preprint arXiv: 2403.01754
44. Liu S Y, Wang C Y, Yin H, Molchanov P, Wang Y C F, Cheng K T, Chen M H. DoRA: weight-decomposed low-rank adaptation. 2024, arXiv preprint arXiv: 2402.09353
45. Qiang R, Zhang R, Xie P. BiLoRA: a bi-level optimization framework for overfitting-resilient low-rank adaptation of large pre-trained models. 2024, arXiv preprint arXiv: 2403.13037
46. Lin Y, Ma X, Chu X, Jin Y, Yang Z, Wang Y, Mei H. LoRA dropout as a sparsity regularizer for overfitting control. 2024, arXiv preprint arXiv: 2404.09610
47. Wang S, Chen L, Jiang J, Xue B, Kong L, Wu C. LoRA meets dropout under a unified framework. 2024, arXiv preprint arXiv: 2403.00812
48. Yang A X, Robeyns M, Wang X, Aitchison L. Bayesian low-rank adaptation for large language models. In: Proceedings of the 12th International Conference on Learning Representations. 2024
49. Qi Z, Tan X, Shi S, Qu C, Xu Y, Qi Y. PILLOW: enhancing efficient instruction fine-tuning via prompt matching. In: Proceedings of 2023 Conference on Empirical Methods in Natural Language Processing: Industry Track. 2023, 471−482
50. Zhang L, Wu J, Zhou D, Xu G. STAR: constraint LoRA with dynamic active learning for data-efficient fine-tuning of large language models. 2024, arXiv preprint arXiv: 2403.01165
51. Wang X, Aitchison L, Rudolph M. LoRA ensembles for large language model fine-tuning. 2023, arXiv preprint arXiv: 2310.00035
52. Zhao Z, Gan L, Wang G, Zhou W, Yang H, Kuang K, Wu F. LoraRetriever: input-aware LoRA retrieval and composition for mixed tasks in the wild. 2024, arXiv preprint arXiv: 2402.09997
53. Smith J S, Cascante-Bonilla P, Arbelle A, Kim D, Panda R, Cox D, Yang D, Kira Z, Feris R, Karlinsky L. ConStruct-VL: data-free continual structured VL concepts learning. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2023, 14994−15004
54. Sun Y, Li M, Cao Y, Wang K, Wang W, Zeng X, Zhao R. To be or not to be? An exploration of continuously controllable prompt engineering. 2023, arXiv preprint arXiv: 2311.09773
55. Zhang J, Chen S, Liu J, He J. Composing parameter-efficient modules with arithmetic operations. 2023, arXiv preprint arXiv: 2306.14870
56. Chitale R, Vaidya A, Kane A, Ghotkar A. Task arithmetic with LoRA for continual learning. 2023, arXiv preprint arXiv: 2311.02428
57. Belofsky J. Token-Level Adaptation of LoRA adapters for downstream task generalization. In: Proceedings of the 6th Artificial Intelligence and Cloud Computing Conference. 2023, 168−172
58. Jiang W, Lin B, Shi H, Zhang Y, Li Z, Kwok J T. Effective and parameter-efficient reusing fine-tuned models. 2023, arXiv preprint arXiv: 2310.01886
59. Asadi N, Beitollahi M, Khalil Y, Li Y, Zhang G, Chen X. Does combining parameter-efficient modules improve few-shot transfer accuracy? 2024, arXiv preprint arXiv: 2402.15414
60. Huang C, Liu Q, Lin B Y, Pang T, Du C, Lin M. LoraHub: efficient cross-task generalization via dynamic LoRA composition. 2023, arXiv preprint arXiv: 2307.13269
61. Yadav P, Choshen L, Raffel C, Bansal M. ComPEFT: compression for communicating parameter efficient updates via sparsification and quantization. 2023, arXiv preprint arXiv: 2311.13171
62. Tang A, Shen L, Luo Y, Zhan Y, Hu H, Du B, Chen Y, Tao D. Parameter-efficient multi-task model fusion with partial linearization. In: Proceedings of the 12th International Conference on Learning Representations. 2024
63. Shen Y, Xu Z, Wang Q, Cheng Y, Yin W, Huang L. Multimodal instruction tuning with conditional mixture of LoRA. In: Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). 2024, 637−648
64. Buehler E L, Buehler M J. X-LoRA: mixture of low-rank adapter experts, a flexible framework for large language models with applications in protein mechanics and molecular design. APL Machine Learning, 2024, 2(2): 026119
65. Yang S, Ali M A, Wang C L, Hu L, Wang D. MoRAL: MoE augmented LoRA for LLMs' lifelong learning. 2024, arXiv preprint arXiv: 2402.11260
66. Dou S, Zhou E, Liu Y, Gao S, Zhao J, Shen W, Zhou Y, Xi Z, Wang X, Fan X, Pu S, Zhu J, Zheng R, Gui T, Zhang Q, Huang X. LoRAMoE: alleviate world knowledge forgetting in large language models via MoE-style plugin. 2023, arXiv preprint arXiv: 2312.09979
67. Gou Y, Liu Z, Chen K, Hong L, Xu H, Li A, Yeung D Y, Kwok J T, Zhang Y. Mixture of cluster-conditional LoRA experts for vision-language instruction tuning. 2023, arXiv preprint arXiv: 2312.12379
68. Liu Q, Wu X, Zhao X, Zhu Y, Xu D, Tian F, Zheng Y. When MOE meets LLMs: parameter efficient fine-tuning for multi-task medical applications. In: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval. 2024, 1104−1114
69. Feng W, Hao C, Zhang Y, Han Y, Wang H. Mixture-of-LoRAs: an efficient multitask tuning method for large language models. In: Proceedings of 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation. 2024, 11371−11380
70. Wang Y, Lin Y, Zeng X, Zhang G. MultiLoRA: democratizing LoRA for better multi-task learning. 2023, arXiv preprint arXiv: 2311.11501
71. Yang Y, Jiang P T, Hou Q, Zhang H, Chen J, Li B. Multi-task dense prediction via mixture of low-rank experts. 2024, arXiv preprint arXiv:

2403.17749

72. Agiza A, Neseem M, Reda S. MTLoRA: low-rank adaptation approach for efficient multi-task learning. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2024, 16196−16205
73. Gao C, Chen K, Rao J, Sun B, Liu R, Peng D, Zhang Y, Guo X, Yang J, Subrahmanian V S. Higher layers need more LoRA experts. 2024, arXiv preprint arXiv: 2402.08562
74. Chen S, Jie Z, Ma L. LLaVA-MoLE: sparse mixture of LoRA experts for mitigating data conflicts in instruction finetuning MLLMs. 2024, arXiv preprint arXiv: 2401.16160
75. Zhu Y, Wichers N, Lin C C, Wang X, Chen T, Shu L, Lu H, Liu C, Luo L, Chen J, Meng L. SiRA: sparse mixture of low rank adaptation. 2023, arXiv preprint arXiv: 2311.09179
76. Chen Z, Wang Z, Wang Z, Liu H, Yin Z, Liu S, Sheng L, Ouyang W, Qiao Y, Shao J. Octavius: mitigating task interference in MLLMs via MoE. 2023, arXiv preprint arXiv: 2311.02684
77. Wen Y, Chaudhuri S. Batched low-rank adaptation of foundation models. In: Proceedings of the Twelfth International Conference on Learning Representations. 2024
78. Wu T, Wang J, Zhao Z, Wong N. Mixture-of-Subspaces in Low-Rank Adaptation. 2024, arXiv preprint arXiv:2406.11909
79. Wu Y, Xiang Y, Huo S, Gong Y, Liang P. LoRA-SP: streamlined partial parameter adaptation for resource efficient fine-tuning of large language models. In: Proceedings of the 3rd International Conference on Algorithms, Microchips, and Network Applications. 2024, 131711Z
80. Zhang L, Zhang L, Shi S, Chu X, Li B. LoRA-FA: memory-efficient low-rank adaptation for large language models fine-tuning. 2023, arXiv preprint arXiv: 2308.03303
81. Liu Z, Kundu S, Li A, Wan J, Jiang L, Beerel P A. AFLoRA: adaptive freezing of low rank adaptation in parameter efficient fine-tuning of large models. 2024, arXiv preprint arXiv: 2403.13269
82. Woo S, Park B, Kim B, Jo M, Kwon S, Jeon D, Lee D. DropBP: accelerating fine-tuning of large language models by dropping backward propagation. 2024, arXiv preprint arXiv: 2402.17812
83. Bałazy K, Banaei M, Aberer K, Tabor J. LoRA-XS: low-rank adaptation with extremely small number of parameters. 2024, arXiv preprint arXiv: 2405.17604
84. Zhou H, Lu X, Xu W, Zhu C, Zhao T, Yang M. LoRA-drop: efficient LoRA parameter pruning based on output evaluation. 2024, arXiv preprint arXiv: 2402.07721
85. Zhang M, Chen H, Shen C, Yang Z, Ou L, Yu X, Zhuang B. LoRAPrune: structured pruning meets low-rank parameter-efficient fine-tuning. In: Proceedings of the Findings of the Association for Computational Linguistics. 2024, 3013−3026
86. Chen T, Ding T, Yadav B, Zharkov I, Liang L. LoRAShear: efficient large language model structured pruning and knowledge recovery. 2023, arXiv preprint arXiv: 2310.18356
87. Zhu Y, Yang X, Wu Y, Zhang W. Parameter-efficient fine-tuning with layer pruning on free-text sequence-to-sequence modeling. 2023, arXiv preprint arXiv: 2305.08285
88. Kopiczko D J, Blankevoort T, Asano Y M. VeRA: vector-based random matrix adaptation. In: Proceedings of the 12th International Conference on Learning Representations. 2024
89. Li Y, Han S, Ji S. VB-LoRA: extreme parameter efficient fine-tuning with vector banks. 2024, arXiv preprint arXiv: 2405.15179
90. Gao Z, Wang Q, Chen A, Liu Z, Wu B, Chen L, Li J. Parameter-efficient fine-tuning with discrete Fourier transform. 2024, arXiv preprint arXiv: 2405.03003
91. Dettmers T, Pagnoni A, Holtzman A, Zettlemoyer L. QLORA: efficient finetuning of quantized LLMs. In: Proceedings of the 37th International Conference on Neural Information Processing Systems. 2023
92. Xu Y, Xie L, Gu X, Chen X, Chang H, Zhang H, Chen Z, Zhang X, Tian Q. QA-LoRA: quantization-aware low-rank adaptation of large language models. In: Proceedings of the 12th International Conference on Learning Representations. 2024
93. Li Y, Yu Y, Liang C, He P, Karampatziakis N, Chen W, Zhao T. LoftQ: LoRA-fine-tuning-aware quantization for large language models. In: Proceedings of the 12th International Conference on Learning Representations. 2024
94. Liao B, Herold C, Khadivi S, Monz C. ApiQ: finetuning of 2-bit quantized large language model. 2024, arXiv preprint arXiv: 2402.05147
95. Jeon H, Kim Y, Kim J J. L4Q: parameter efficient quantization-aware training on large language models via LoRA-wise LSQ. 2024, arXiv preprint arXiv: 2402.04902
96. Ye Z, Li D, Tian J, Lan T, Zuo J, Duan L, Lu H, Jiang Y, Sha J, Zhang K, Tang M. ASPEN: high-throughput LoRA fine-tuning of large language models with a single GPU. 2023, arXiv preprint arXiv: 2312.02515
97. Chen L, Ye Z, Wu Y, Zhuo D, Ceze L, Krishnamurthy A. Punica: multi-tenant LoRA serving. In: Proceedings of the Seventh Annual Conference on Machine Learning and Systems. 2024, 1−13
98. Sheng Y, Cao S, Li D, Hooper C, Lee N, Yang S, Chou C, Zhu B, Zheng L, Keutzer K, Gonzalez J E, Stoica I. S-LoRA: serving thousands of concurrent LoRA adapters. 2023, arXiv preprint arXiv: 2311.03285
99. Li S, Lu H, Wu T, Yu M, Weng Q, Chen X, Shan Y, Yuan B, Wang W. CaraServe: CPU-assisted and rank-aware LoRA serving for generative LLM inference. 2024, arXiv preprint arXiv: 2401.11240
100. Babakniya S, Elkordy A R, Ezzeldin Y H, Liu Q, Song K B, El-Khamy M, Avestimehr S. SLoRA: federated parameter efficient fine-tuning of language models. 2023, arXiv preprint arXiv: 2308.06522
101. Yan Y, Tang S, Shi Z, Yang Q. FeDeRA: efficient fine-tuning of language models in federated learning leveraging weight decomposition. 2024, arXiv preprint arXiv: 2404.18848
102. Sun Y, Li Z, Li Y, Ding B. Improving LoRA in privacy-preserving federated learning. In: Proceedings of the 12th International Conference on Learning Representations. 2024
103. Wu P, Li K, Wang T, Wang F. FedMS: federated learning with mixture of sparsely activated foundations models. 2023, arXiv preprint arXiv: 2312.15926
104. Bai J, Chen D, Qian B, Yao L, Li Y. Federated fine-tuning of large language models under heterogeneous language tasks and client resources. 2024, arXiv preprint arXiv: 2402.11505
105. Cho Y J, Liu L, Xu Z, Fahrezi A, Barnes M, Joshi G. Heterogeneous LoRA for federated fine-tuning of on-device foundation models. In: Proceedings of the International Workshop on Federated Learning in the Age of Foundation Models in Conjunction with NeurIPS. 2023
106. Yi L, Yu H, Wang G, Liu X, Li X. pFedLoRA: model-heterogeneous personalized federated learning with LoRA tuning. 2023, arXiv preprint arXiv: 2310.13283
107. Huang W, Wang Y, Cheng A, Zhou A, Yu C, Wang L. A fast, performant, secure distributed training framework for large language model. 2024, arXiv preprint arXiv: 2401.09796
108. Wang Y, Lin Y, Zeng X, Zhang G. PrivateLoRA for efficient privacy preserving LLM. 2023, arXiv preprint arXiv: 2311.14030
109. Zhang Y, Wang M, Wu Y, Tiwari P, Li Q, Wang B, Qin J. DialogueLLM: context and emotion knowledge-tuned large language models for emotion recognition in conversations. 2024, arXiv preprint arXiv: 2310.11374
110. Li Z, Li X, Liu Y, Xie H, Li J, Wang F L, Li Q, Zhong X. Label supervised LLaMA finetuning. 2023, arXiv preprint arXiv:

2310.01208

111. Bornheim T, Grieger N, Blaneck P G, Bialonski S. Speaker attribution in German parliamentary debates with QLoRA-adapted large language models. 2024, arXiv preprint arXiv: 2309.09902
112. Xue L, Zhang D, Dong Y, Tang J. AutoRE: document-level relation extraction with large language models. 2024, arXiv preprint arXiv: 2403.14888
113. Alves D M, Guerreiro N M, Alves J, Pombal J, Rei R, de Souza J G C, Colombo P, Martins A F T. Steering large language models for machine translation with finetuning and in-context learning. In: Proceedings of the Findings of the Association for Computational Linguistics. 2023, 11127−11148
114. Zheng J, Hong H, Wang X, Su J, Liang Y, Wu S. Fine-tuning large language models for domain-specific machine translation. 2024, arXiv preprint arXiv: 2402.15061
115. Mujadia V, Urlana A, Bhaskar Y, Pavani P A, Shravya K, Krishnamurthy P, Sharma D M. Assessing translation capabilities of large language models involving English and Indian languages. 2023, arXiv preprint arXiv: 2311.09216
116. Zhang Y, Wang J, Yu L C, Xu D, Zhang X. Personalized LoRA for human-centered text understanding. In: Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence. 2024, 19588−19596
117. Liu Y, An C, Qiu X. *Y*-tuning: an efficient tuning paradigm for large-scale pre-trained models via label representation learning. Frontiers of Computer Science, 2024, 18(4): 184320
118. Liu S, Keung J, Yang Z, Liu F, Zhou Q, Liao Y. Delving into parameter-efficient fine-tuning in code change learning: an empirical study. In: Proceedings of the IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER). 2024, 465−476
119. Guo Y, Gao X, Jiang B. An empirical study on JIT defect prediction based on BERT-style model. 2024, arXiv preprint arXiv: 2403.11158
120. Ayupov S, Chirkova N. Parameter-efficient finetuning of transformers for source code. 2022, arXiv preprint arXiv: 2212.05901
121. Silva A, Fang S, Monperrus M. RepairLLaMA: efficient representations and fine-tuned adapters for program repair. 2023, arXiv preprint arXiv: 2312.15698
122. Roberson R, Kaki G, Trivedi A. Analyzing the effectiveness of large language models on text-to-SQL synthesis. 2024, arXiv preprint arXiv: 2401.12379
123. Pan J, Sadé A, Kim J, Soriano E, Sole G, Flamant S. SteloCoder: a decoder-only LLM for multi-language to python code translation. 2023, arXiv preprint arXiv: 2310.15539
124. Sidahmed H, Phatale S, Hutcheson A, Lin Z, Chen Z, Yu Z, Jin J, Komarytsia R, Ahlheim C, Zhu Y, Chaudhary S, Li B, Ganesh S, Byrne B, Hoffmann J, Mansoor H, Li W, Rastogi A, Dixon L. PERL: parameter efficient reinforcement learning from human feedback. 2024, arXiv preprint arXiv: 2403.10704
125. Santacroce M, Lu Y, Yu H, Li Y, Shen Y. Efficient RLHF: reducing the memory usage of PPO. 2023, arXiv preprint arXiv: 2309.00754
126. Sun S, Gupta D, Iyyer M. Exploring the impact of low-rank adaptation on the performance, efficiency, and regularization of RLHF. 2023, arXiv preprint arXiv: 2309.09055
127. Quan S. DMoERM: recipes of mixture-of-experts for effective reward modeling. 2024, arXiv preprint arXiv: 2403.01197
128. Zhang S, Chen Z, Chen S, Shen Y, Sun Z, Gan C. Improving reinforcement learning from human feedback with efficient reward model ensemble. 2024, arXiv preprint arXiv: 2401.16635
129. Zhai Y, Zhang H, Lei Y, Yu Y, Xu K, Feng D, Ding B, Wang H. Uncertainty-penalized reinforcement learning from human feedback with diverse reward LoRA ensembles. 2023, arXiv preprint arXiv: 2401.00243
130. Yang A X, Robeyns M, Coste T, Shi Z, Wang J, Bou-Ammar H, Aitchison L. Bayesian reward models for LLM alignment. 2024, arXiv preprint arXiv: 2402.13210
131. Daxberger E, Kristiadi A, Immer A, Eschenhagen R, Bauer M, Hennig P. Laplace redux-effortless bayesian deep learning. Advances in Neural Information Processing Systems. 2021
132. Tran H, Yang Z, Yao Z, Yu H. BioInstruct: instruction tuning of large language models for biomedical natural language processing. 2023, arXiv preprint arXiv: 2310.19975
133. Gema A P, Minervini P, Daines L, Hope T, Alex B. Parameter-efficient fine-tuning of LLaMA for the clinical domain. 2023, arXiv preprint arXiv: 2307.03042
134. Toma A, Lawler P R, Ba J, Krishnan R G, Rubin B B, Wang B. Clinical camel: an open-source expert-level medical language model with dialogue-based knowledge encoding. 2023, arXiv preprint arXiv: 2305.12031
135. Suri K, Mishra P, Saha S, Singh A. Suryakiran at MEDIQA-Sum 2023: leveraging LoRA for clinical dialogue summarization. In: Proceedings of the Working Notes of the Conference and Labs of the Evaluation Forum. 2023, 1720−1735
136. Ji Y, Yu Z, Wang Y. Assertion detection large language model in-context learning LoRA fine-tuning. 2024, arXiv preprint arXiv: 2401.17602
137. Wang R, Duan Y, Lam C, Chen J, Xu J, Chen H, Liu X, Pang P C I, Tan T. IvyGPT: InteractiVe Chinese pathway language model in medical domain. In: Proceedings of the 3rd CAAI International Conference on Artificial Intelligence. 2024, 378−382
138. Bhatti A, Parmar S, Lee S. SM70: a large language model for medical devices. 2023, arXiv preprint arXiv: 2312.06974
139. Konstantinidis T, Iacovides G, Xu M, Constantinides T G, Mandic D. FinLlama: financial sentiment classification for algorithmic trading applications. 2024, arXiv preprint arXiv: 2403.12285
140. Pavlyshenko B M. Financial news analytics using fine-tuned llama 2 GPT model. 2023, arXiv preprint arXiv: 2308.13032
141. Liu X Y, Wang G, Yang H, Zha D. FinGPT: democratizing internet-scale data for financial large language models. 2023, arXiv preprint arXiv: 2307.10485
142. Li J, Lei Y, Bian Y, Cheng D, Ding Z, Jiang C. RA-CFGPT: Chinese financial assistant with retrieval-augmented large language model. Frontiers of Computer Science, 2024, 18(5): 185350
143. Zhou X, Sun Z, Li G. DB-GPT: large language model meets database. Data Science and Engineering, 2024, 9(1): 102−111
144. Li S. DiffStyler: diffusion-based localized image style transfer. 2024, arXiv preprint arXiv: 2403.18461
145. Frenkel Y, Vinker Y, Shamir A, Cohen-Or D. Implicit style-content separation using B-LoRA. 2024, arXiv preprint arXiv: 2403.14572
146. Liu Y, Yu C, Shang L, He Y, Wu Z, Wang X, Xu C, Xie H, Wang W, Zhao Y, Zhu L, Cheng C, Chen W, Yao Y, Zhou W, Xu J, Wang Q, Chen Y, Xie X, Sun B. FaceChain: a playground for human-centric artificial intelligence generated content. 2023, arXiv preprint arXiv: 2308.14256
147. Liao Q, Xia G, Wang Z. Calliffusion: Chinese calligraphy generation and style transfer with diffusion modeling. 2023, arXiv preprint arXiv: 2305.19124
148. Shrestha S, Sripada V S S, Venkataramanan A. Style transfer to Calvin and Hobbes comics using stable diffusion. 2023, arXiv preprint arXiv: 2312.03993
149. Li L, Zeng H, Yang C, Jia H, Xu D. Block-wise LoRA: revisiting fine-grained LoRA for effective personalization and stylization in text-to-image generation. 2024, arXiv preprint arXiv: 2403.07500
150. Kong Z, Zhang Y, Yang T, Wang T, Zhang K, Wu B, Chen G, Liu W,

Luo W. OMG: occlusion-friendly personalized multi-concept generation in diffusion models. 2024, arXiv preprint arXiv: 2403.10983

151. Shi J, Hua H. Space narrative: generating images and 3D scenes of Chinese garden from text using deep learning. In: Proceedings of the xArch-Creativity in the Age of Digital Reproduction Symposium. 2024, 236−243
152. Jin Z, Song Z. Generating coherent comic with rich story using ChatGPT and stable diffusion. 2023, arXiv preprint arXiv: 2305.11067
153. Wang H, Xiang X, Fan Y, Xue J H. Customizing 360-degree panoramas through text-to-image diffusion models. In: Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision. 2024, 4921−4931
154. Guo J, Xu X, Pu Y, Ni Z, Wang C, Vasu M, Song S, Huang G, Shi H. Smooth diffusion: crafting smooth latent spaces in diffusion models. 2023, arXiv preprint arXiv: 2312.04410
155. Cheng J, Xie P, Xia X, Li J, Wu J, Ren Y, Li H, Xiao X, Zheng M, Fu L. ResAdapter: domain consistent resolution adapter for diffusion models. 2024, arXiv preprint arXiv: 2403.02084
156. Smith J S, Hsu Y C, Kira Z, Shen Y, Jin H. Continual diffusion with STAMINA: STack-and-mask INcremental adapters. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2024, 1744−1754
157. Sun J, Fu D, Hu Y, Wang S, Rassin R, Juan D C, Alon D, Herrmann C, van Steenkiste S, Krishna R, Rashtchian C. Dreamsync: aligning text-to-image generation with image understanding feedback. In: Proceedings of the Synthetic Data for Computer Vision Workshop@ CVPR 2024. 2023
158. Wang Z, Wang X, Xie L, Qi Z, Shan Y, Wang W, Luo P. StyleAdapter: a single-pass LoRA-free model for stylized image generation. 2023, arXiv preprint arXiv: 2309.01770
159. Gu Y, Wang X, Wu J Z, Shi Y, Chen Y, Fan Z, Xiao W, Zhao R, Chang S, Wu W, Ge Y, Shan Y, Shou M Z. Mix-of-show: decentralized low-rank adaptation for multi-concept customization of diffusion models. In: Proceedings of the 37th International Conference on Neural Information Processing Systems. 2023
160. Luo S, Tan Y, Patil S, Gu D, von Platen P, Passos A, Huang L, Li J, Zhao H. LCM-LoRA: a universal stable-diffusion acceleration module. 2023, arXiv preprint arXiv: 2311.05556
161. Golnari P A. LoRA-enhanced distillation on guided diffusion models. 2023, arXiv preprint arXiv: 2312.06899
162. Ren Y, Zhou Y, Yang J, Shi J, Liu D, Liu F, Kwon M, Shrivastava A. Customize-A-video: one-shot motion customization of text-to-video diffusion models. 2024, arXiv preprint arXiv: 2402.14780
163. Deng Y, Wang R, Zhang Y, Tai Y W, Tang C K. DragVideo: interactive drag-style video editing. 2023, arXiv preprint arXiv: 2312.02216
164. Yang S, Zhou Y, Liu Z, Loy C C. Rerender A video: zero-shot text-guided video-to-video translation. In: Proceedings of the SIGGRAPH Asia 2023 Conference Papers. 2023, 95
165. Khandelwal A. InFusion: inject and attention fusion for multi concept zero-shot text-based video editing. In: Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops. 2023, 3009−3018
166. Blattmann A, Dockhorn T, Kulal S, Mendelevitch D, Kilian M, Lorenz D, Levi Y, English Z, Voleti V, Letts A, Jampani V, Rombach R. Stable video diffusion: scaling latent video diffusion models to large datasets. 2023, arXiv preprint arXiv: 2311.15127
167. Guo Y, Yang C, Rao A, Liang Z, Wang Y, Qiao Y, Agrawala M, Lin D, Dai B. AnimateDiff: animate your personalized text-to-image diffusion models without specific tuning. In: Proceedings of the 12th International Conference on Learning Representations. 2024
168. Huang T, Zeng Y, Zhang Z, Xu W, Xu H, Xu S, Lau R W H, Zuo W. DreamControl: control-based text-to-3D generation with 3D self-prior. 2023, arXiv preprint arXiv: 2312.06439
169. Ma Y, Fan Y, Ji J, Wang H, Sun X, Jiang G, Shu A, Ji R. X-dreamer: creating high-quality 3D content by bridging the domain gap between text-to-2D and text-to-3D generation. 2023, arXiv preprint arXiv: 2312.00085
170. Yu K, Liu J, Feng M, Cui M, Xie X. Boosting3D: high-fidelity image-to-3D by boosting 2D diffusion prior to 3D prior with progressive learning. 2023, arXiv preprint arXiv: 2311.13617
171. Yoo S, Kim K, Kim V G, Sung M. As-plausible-as-possible: plausibility-aware mesh deformation using 2D diffusion priors. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2024, 4315−4324
172. Zhang Y, Xu Q, Zhang L. DragTex: generative point-based texture editing on 3D mesh. 2024, arXiv preprint arXiv: 2403.02217
173. Ding H, Gao J, Yuan Y, Wang Q. SamLP: a customized segment anything model for license plate detection. 2024, arXiv preprint arXiv: 2401.06374
174. Ye Z, Lovell L, Faramarzi A, Ninic J. SAM-based instance segmentation models for the automation of structural damage detection. 2024, arXiv preprint arXiv: 2401.15266
175. Na S, Guo Y, Jiang F, Ma H, Huang J. Segment any cell: a SAM-based auto-prompting fine-tuning framework for nuclei segmentation. 2024, arXiv preprint arXiv: 2401.13220
176. Chen X, Wang C, Ning H, Li S, Shen M. SAM-OCTA: prompting segment-anything for OCTA image segmentation. 2023, arXiv preprint arXiv: 2310.07183
177. Feng W, Zhu L, Yu L. Cheap lunch for medical image segmentation by fine-tuning SAM on few exemplars. 2023, arXiv preprint arXiv: 2308.14133
178. Zhang K, Liu D. Customized segment anything model for medical image segmentation. 2023, arXiv preprint arXiv: 2304.13785
179. Wang A, Islam M, Xu M, Zhang Y, Ren H. SAM meets robotic surgery: an empirical study on generalization, robustness and adaptation. In: Proceedings of the International Conference on Medical Image Computing and Computer-Assisted Intervention. 2023, 234−244
180. Lin L, Fan H, Zhang Z, Wang Y, Xu Y, Ling H. Tracking meets LoRA: faster training, larger model, stronger performance. 2024, arXiv preprint arXiv: 2403.05231
181. Kong C, Li H, Wang S. Enhancing general face forgery detection via vision transformer with low-rank adaptation. In: Proceedings of the 6th International Conference on Multimedia Information Processing and Retrieval. 2023, 102−107
182. Chen Z, Huang H, Andrusenko A, Hrinchuk O, Puvvada K C, Li J, Ghosh S, Balam J, Ginsburg B. SALM: speech-augmented language model with in-context learning for speech recognition and translation. In: Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). 2024, 13521−13525
183. Dong X, Zhang P, Zang Y, Cao Y, Wang B, Ouyang L, Wei X, Zhang S, Duan H, Cao M, Zhang W, Li Y, Yan H, Gao Y, Zhang X, Li W, Li J, Chen K, He C, Zhang X, Qiao Y, Lin D, Wang J. InternLM-XComposer2: mastering free-form text-image composition and comprehension in vision-language large model. 2024, arXiv preprint arXiv: 2401.16420
184. Ye Q, Xu H, Xu G, Ye J, Yan M, Zhou Y, Wang J, Hu A, Shi P, Shi Y, Li C, Xu Y, Chen H, Tian J, Qian Q, Zhang J, Huang F, Zhou J. mPLUG-Owl: modularization empowers large language models with multimodality. 2023, arXiv preprint arXiv: 2304.14178
185. Lee B K, Park B, Kim C W, Ro Y M. CoLLaVO: crayon large language and vision mOdel. 2024, arXiv preprint arXiv: 2402.11248

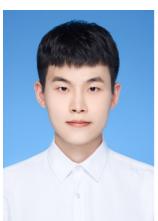
186. Yeo J H, Han S, Kim M, Ro Y M. Where visual speech meets language: VSP-LLM framework for efficient and context-aware visual speech processing. 2024, arXiv preprint arXiv: 2402.15151
187. Liu Z, Li S, Luo Y, Fei H, Cao Y, Kawaguchi K, Wang X, Chua T S. MolCA: molecular graph-language modeling with cross-modal projector and uni-modal adapter. In: Proceedings of 2023 Conference on Empirical Methods in Natural Language Processing. 2023, 15623−15638
188. Ren Y, Chen Y, Liu S, Wang B, Yu H, Cui Z. TPLLM: a traffic prediction framework based on pretrained large language models. 2024, arXiv preprint arXiv: 2403.02221
189. Aghajanyan A, Gupta S, Zettlemoyer L. Intrinsic dimensionality explains the effectiveness of language model fine-tuning. In: Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing. 2021, 7319−7328
190. Fomenko V, Yu H, Lee J, Hsieh S, Chen W. A note on LoRA. 2024, arXiv preprint arXiv: 2404.05086
191. Bershatsky D, Cherniuk D, Daulbaev T, Mikhalev A, Oseledets I. LoTR: low tensor rank weight adaptation. 2024, arXiv preprint arXiv: 2402.01376
192. Edalati A, Tahaei M, Kobyzev I, Nia V P, Clark J J, Rezagholizadeh M. KronA: parameter efficient tuning with kronecker adapter. 2022, arXiv preprint arXiv: 2212.10650
193. He X, Li C, Zhang P, Yang J, Wang X E. Parameter-efficient model adaptation for vision transformers. In: Proceedings of the 37th AAAI Conference on Artificial Intelligence. 2023, 817−825
194. Zhao Z, Gan L, Wang G, Hu Y, Shen T, Yang H, Kuang K, Wu F. Retrieval-augmented mixture of lora experts for uploadable machine learning. 2024 , arXiv preprint arXiv:2406.16989.
195. Mahabadi R K, Henderson J, Ruder S. COMPACTER: efficient low-rank hypercomplex adapter layers. In: Proceedings of the 35th International Conference on Neural Information Processing Systems. 2021, 79
196. Liao B, Meng Y, Monz C. Parameter-efficient fine-tuning without introducing new latency. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics. 2023, 4242−4260
197. Hendrycks D, Burns C, Basart S, Zou A, Mazeika M, Song D, Steinhardt J. Measuring massive multitask language understanding. In: Proceedings of the 9th International Conference on Learning Representations. 2021
198. He J, Zhou C, Ma X, Berg-Kirkpatrick T, Neubig G. Towards a unified view of parameter-efficient transfer learning. In: Proceedings of the 10th International Conference on Learning Representations. 2022
199. Geshkovski B, Letrouit C, Polyanskiy Y, Rigollet P. A mathematical perspective on transformers. 2023, arXiv preprint arXiv: 2312.10794
200. Geshkovski B, Letrouit C, Polyanskiy Y, Rigollet P. The emergence of clusters in self-attention dynamics. In: Proceedings of the 37th International Conference on Neural Information Processing Systems. 2023
201. Sander M E, Ablin P, Blondel M, Peyré G. Sinkformers: transformers with doubly stochastic attention. In: Proceedings of the 25th International Conference on Artificial Intelligence and Statistics. 2022, 3515−3530
202. Jacot A, Gabriel F, Hongler C. Neural tangent kernel: convergence and generalization in neural networks. In: Proceedings of the 32nd International Conference on Neural Information Processing Systems. 2018, 8580−8589
203. Touvron H, Martin L, Stone K, Albert P, Almahairi A, Babaei Y, Bashlykov N, Batra S, Bhargava P, Bhosale S, Bikel D, Blecher L, Canton Ferrer C, Chen M, Cucurull G, Esiobu D, Fernandes J, Fu J, Fu W, Fuller B, Gao C, Goswami V, Goyal N, Hartshorn A, Hosseini S, Hou R, Inan H, Kardas M, Kerkez V, Khabsa M, Kloumann I, Korenev A, Koura P S, Lachaux M A, Lavril T, Lee J, Liskovich D, Lu Y, Mao Y, Martinet X, Mihaylov T, Mishra P, Molybog I, Nie Y, Poulton A, Reizenstein J, Rungta R, Saladi K, Schelten A, Silva R, Smith E M, Subramanian R, Tan X E, Tang B, Taylor R, Williams A, Kuan J X, Xu P, Yan Z, Zarov I, Zhang Y, Fan A, Kambadur M, Narang S, Rodriguez A, Stojnic R, Edunov S, Scialom T. Llama 2: open foundation and fine-tuned chat models. 2023, arXiv preprint arXiv: 2307.09288
204. Chang Y, Chang Y, Wu Y. Bias-Aware Low-Rank Adaptation: Mitigating Catastrophic Inheritance of Large Language Models. 2024 , arXiv preprint arXiv:2408.04556
205. Zhao J, Zhang Z, Chen B, Wang Z, Anandkumar A, Tian Y. Galore: memory-efficient LLM training by gradient low-rank projection. 2024, arXiv preprint arXiv: 2403.03507
206. Biderman D, Ortiz J G, Portes J, Paul M, Greengard P, Jennings C, King D, Havens S, Chiley V, Frankle J, Blakeney C, Cunningham J P. LoRA learns less and forgets less. 2024, arXiv preprint arXiv: 2405.09673
207. Han A, Li J, Huang W, Hong M, Takeda A, Jawanpuria P, Mishra B. SLTrain: a sparse plus low-rank approach for parameter and memory efficient pretraining. 2024, arXiv preprint arXiv: 2406.02214
208. Sui Y, Yin M, Gong Y, Xiao J, Phan H, Yuan B. ELRT: efficient low-rank training for compact convolutional neural networks. 2024, arXiv preprint arXiv: 2401.10341
209. Meng X, Dai D, Luo W, Yang Z, Wu S, Wang X, Wang P, Dong Q, Chen L, Sui Z. PeriodicLoRA: breaking the low-rank bottleneck in LoRA optimization. 2024, arXiv preprint arXiv: 2402.16141
210. Frank M, Wolfe P. An algorithm for quadratic programming. Naval Research Logistics Quarterly, 1956, 3(1-2): 95−110
211. Rajabzadeh H, Valipour M, Zhu T, Tahaei M, Kwon HJ, Ghodsi A, Chen B, Rezagholizadeh M. Qdylora: Quantized dynamic low-rank adaptation for efficient large language model tuning. 2024 , arXiv preprint arXiv:2402.10462
212. Elsken T, Metzen J H, Hutter F. Neural architecture search: a survey. The Journal of Machine Learning Research, 2019, 20(1): 1997−2017
213. Liu Y, Ott M, Goyal N, Du J, Joshi M, Chen D, Levy O, Lewis M, Zettlemoyer L, Stoyanov V. RoBERTa: a robustly optimized BERT pretraining approach. 2019, arXiv preprint arXiv: 1907.11692
214. Wang A, Singh A, Michael J, Hill F, Levy O, Bowman S R. GLUE: a multi-task benchmark and analysis platform for natural language understanding. In: Proceedings of 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP. 2018, 353−355
215. Renduchintala A, Konuk T, Kuchaiev O. Tied-LoRA: enhancing parameter efficiency of LoRA with weight tying. In: Proceedings of 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies. 2024, 8694−8705
216. Hansen N, Ostermeier A. Adapting arbitrary normal mutation distributions in evolution strategies: the covariance matrix adaptation. In: Proceedings of the IEEE International Conference on Evolutionary Computation. 1996, 312−317
217. Ye M, Fang X, Du B, Yuen P C, Tao D. Heterogeneous federated learning: state-of-the-art and research challenges. ACM Computing Surveys, 2024, 56(3): 79
218. Liu X Y, Zhu R, Zha D, Gao J, Zhong S, White M, Qiu M. Differentially private low-rank adaptation of large language model using federated learning. 2023, arXiv preprint arXiv: 2312.17493



Yuren Mao received his PhD degree under the supervision of Prof. Xuemin Lin in computer science from University of New South Wales, Australia in 2022. He is currently an assistant professor with the School of Software Technology, Zhejiang University, China. His current research interests include Large Language Models and its applications in Data Intelligence.



Yuhang Ge is currently working toward his PhD degree in the School of Software Technology at Zhejiang University, China. His research interests include Large Language Models and Data Management.



Yijiang Fan is currently studying as a master's student in the School of Software Technology at Zhejiang University, China. His research interests include Large Language Models and collaborative inference.



Wenyi Xu is currently studying as a master's student in the School of Software Technology at Zhejiang University, China. His research interests include Multimodal Large Models and RAG.



Yu Mi is currently studying as a master's student in the School of Software Technology at Zhejiang University, China. Her research interests include Large Language Models and AI for science.



Zhonghao Hu is currently studying as a master's student in the School of Software Technology at Zhejiang University, China. His research interests include Large Language Models and data discovery.



Yunjun Gao received the PhD degree in computer science from Zhejiang University, China, in 2008. He is currently a professor in the College of Computer Science and Technology, Zhejiang University, China. His research interests include Database, Big Data Management and Analytics, and AI interaction with DB technology.